



PowerChat

Técnico Superior Desarrollo de aplicaciones Web / Online

Javier García Manotas

Tutor del TFG: Raúl Albiol

ABSTRACT

PowerChat es una aplicación web diseñada para la gestión profesional de conversaciones de WhatsApp en entornos empresariales multiagente. El sistema permite que varios agentes trabajen simultáneamente sobre conversaciones entrantes, con actualización en tiempo real, control de estados y sistema de asignaciones. Se ha desarrollado utilizando Node.js con Express en el backend, Vue 3 con Composition API en el frontend, MySQL como base de datos relacional gestionada a través de Prisma ORM, y Socket.IO para la comunicación en tiempo real mediante WebSockets. La aplicación implementa un sistema completo de autenticación basado en JWT, control de acceso por roles (Administrador, Supervisor y Agente), integración con proveedores de mensajería externos mediante webhooks, un panel de estadísticas con visualizaciones gráficas, un sistema de etiquetas personalizables para clasificar conversaciones, análisis automático de sentimientos sobre mensajes entrantes, y un asistente de datos con inteligencia artificial que permite realizar consultas en lenguaje natural sobre los datos del sistema. El resultado es una herramienta funcional, segura y responsiva que aborda los elementos fundamentales de las plataformas profesionales de atención al cliente a través de WhatsApp.

Palabras clave: WhatsApp, multiagente, atención al cliente, tiempo real, Vue, Node.js, análisis de sentimiento, asistente de IA.

PowerChat is a web application designed for the professional management of WhatsApp conversations in multi-agent business environments. The system enables multiple agents to work simultaneously on incoming conversations, with real-time updates, status control, and an assignment system. It was developed using Node.js with Express on the backend, Vue 3 with Composition API on the frontend, MySQL as the relational database managed through Prisma ORM, and Socket.IO for real-time communication via WebSockets. The application implements a complete JWT-based authentication system, role-based access control

(Administrator, Supervisor, and Agent), integration with external messaging providers through webhooks, a statistics dashboard with graphical visualizations, a customizable label system for classifying conversations, automatic sentiment analysis on incoming messages, and an AI-powered data assistant that enables natural language queries over the system's data. The result is a functional, secure, and responsive tool that addresses the fundamental elements of professional customer service platforms through WhatsApp.

Keywords: WhatsApp, multi-agent, customer service, real-time, Vue, Node.js, sentiment analysis, AI assistant.

ÍNDICES

ABSTRACT	2
ÍNDICES	4
INTRODUCCIÓN	6
JUSTIFICACIÓN DEL PROYECTO	7
Estado del arte y comparativa	7
Público objetivo	8
OBJETIVOS.....	9
R01 – El sistema debe permitir la autenticación y gestión de usuarios.....	9
R02 – El sistema debe gestionar conversaciones de WhatsApp	10
R03 – El sistema debe permitir el envío y recepción de mensajes en tiempo real.....	11
R04 – El sistema debe recibir eventos externos mediante webhooks	12
R05 – El sistema debe ofrecer un panel de estadísticas	12
R06 – La interfaz debe ser responsiva y adaptarse a múltiples dispositivos.....	12
R07 – El sistema debe permitir clasificar conversaciones mediante etiquetas	13
R08 – El sistema debe analizar el sentimiento de los mensajes entrantes	13
R09 – El sistema debe ofrecer un asistente de datos con inteligencia artificial	14
R10 – El sistema debe gestionar el estado de lectura de las conversaciones por usuario	15
DESCRIPCIÓN	16
Arquitectura de la solución	16
Casos de uso	17
DISEÑOS.....	19
Diagrama Entidad-Relación	19
Diagrama de la base de datos.....	21
Interfaces de usuario.....	23
Pantalla de inicio de sesión	23
Bandeja de conversaciones	24
Cabecera de conversación y asignación	25
Burbujas de mensaje.....	26
Panel de estadísticas (Dashboard).....	28
Gestión de usuarios	29
Perfil de usuario	30
Gestión de etiquetas	31
Asistente de datos (IA)	32
TECNOLOGÍA	34
Backend.....	34
Frontend	36
Herramientas de desarrollo	37

Despliegue	37
Frontend: Vercel.....	37
Backend y base de datos: Railway.....	39
Modelo de lenguaje: RunPod	41
DNS y seguridad: Cloudflare	43
Arquitectura de despliegue completa.....	44
METODOLOGÍA	46
Metodología utilizada: Desarrollo iterativo incremental.....	46
Planificación y presupuesto de horas	49
README y GIT	50
TRABAJOS FUTUROS.....	51
CONCLUSIONES	52
REFERENCIAS	54

INTRODUCCIÓN

PowerChat es una aplicación web diseñada para la gestión profesional de conversaciones de WhatsApp en empresas. En un contexto donde WhatsApp se ha consolidado como uno de los canales de comunicación más utilizados entre empresas y clientes, surge la necesidad de herramientas que permitan gestionar estas conversaciones de forma estructurada, especialmente cuando intervienen varios empleados o agentes de atención al cliente.

El sistema permite que varios agentes trabajen simultáneamente sobre conversaciones entrantes, ofreciendo actualización en tiempo real, control de estados de las conversaciones (abierta, pendiente, cerrada) y un sistema completo de asignaciones entre agentes. Todo ello desde una interfaz web responsiva que se adapta a dispositivos de escritorio, tableta y móvil.

Los principales requisitos que debe cumplir el proyecto son los siguientes: permitir la gestión multiagente de conversaciones de WhatsApp, ofrecer comunicación en tiempo real mediante WebSockets, implementar un sistema de autenticación y autorización basado en roles (Administrador, Supervisor y Agente), integrar un proveedor externo de mensajería a través de webhooks, proporcionar un panel de estadísticas para la supervisión del rendimiento, implementar un sistema de etiquetas personalizables para clasificar conversaciones, incorporar análisis automático de sentimientos sobre los mensajes entrantes, ofrecer un asistente de datos con inteligencia artificial para consultas en lenguaje natural, y garantizar una experiencia de usuario fluida y responsiva en todos los dispositivos.

El objetivo es desarrollar una herramienta similar a las plataformas profesionales de atención al cliente como Whanmo, ManyContacts o Respond.io, pero centrada exclusivamente en WhatsApp y con un enfoque técnico académico que demuestre las competencias adquiridas durante el Grado Superior en Desarrollo de Aplicaciones Web.

JUSTIFICACIÓN DEL PROYECTO

La idea del proyecto surge de la experiencia profesional del autor en una empresa que desarrolla una plataforma similar (whanmo.com). Esta experiencia ha permitido conocer de primera mano la creciente importancia de WhatsApp como canal de comunicación empresarial y las carencias que aún existen en muchas organizaciones a la hora de gestionarlo de forma profesional.

Actualmente existen diversas herramientas en el mercado que permiten la gestión multiagente de conversaciones de WhatsApp. Entre las más destacadas se encuentran:

Estado del arte y comparativa

Whanmo (Whanmo, s. f.) (whanmo.com): plataforma SaaS española especializada en la gestión de WhatsApp para equipos comerciales. Ofrece bandeja compartida, asignación automática, etiquetas y seguimiento de conversaciones. Orientada a PYMEs con planes de pago mensual.

ManyContacts: herramienta que permite gestionar WhatsApp desde un CRM compartido. Integra funciones de embudo de ventas, notas internas y sistema de tareas. Orientada a equipos de ventas con necesidades de seguimiento comercial.

Respond.io: plataforma omnicanal que soporta múltiples canales de mensajería (WhatsApp, Telegram, Facebook Messenger, etc.). Ofrece automatizaciones avanzadas, chatbots y análisis detallados. Orientada a empresas medianas y grandes con necesidades multicanal.

Sin embargo, muchas empresas todavía gestionan WhatsApp de forma poco estructurada cuando intervienen varios empleados, utilizando dispositivos compartidos o cuentas sin control interno. Esto genera problemas de organización, reparto de trabajo y trazabilidad.

PowerChat propone una implementación técnica que aborda los elementos fundamentales de estas plataformas profesionales: gestión estructurada de conversaciones, asignación entre agentes, control de estados y sincronización en tiempo real. A diferencia de las soluciones comerciales, PowerChat se desarrolla como un proyecto académico de código abierto que puede servir como base para implementaciones personalizadas.

Público objetivo

El sistema está dirigido a empresas que utilizan WhatsApp como canal de comunicación y necesitan organizar la gestión entre varios agentes. Aporta especial valor en organizaciones de tres o más empleados, donde la coordinación y el reparto de conversaciones requieren una herramienta estructurada. Es aplicable a distintos sectores, dado el uso generalizado de WhatsApp en el entorno empresarial español e internacional.

OBJETIVOS

A continuación, se presenta el desglose RFTP (Requisitos, Funciones, Tareas y Pruebas) del proyecto PowerChat. Este desglose permite entender de forma jerárquica qué debe hacer el sistema, cómo se descompone cada requisito en funcionalidades concretas, qué tareas de implementación implica cada una y cómo se verifica su correcto funcionamiento.

R01 – El sistema debe permitir la autenticación y gestión de usuarios

R01F01 – El usuario debe poder iniciar sesión con email y contraseña.

R01F01T01 – Implementar endpoint POST /auth/login con validación de credenciales y generación de token JWT (expiración 24h).

R01F01T01P01 – Verificar login correcto con credenciales válidas y recepción de token.

R01F01T01P02 – Verificar rechazo con credenciales inválidas y mensaje de error apropiado.

R01F01T02 – Implementar rate limiting de 3 intentos cada 15 segundos para prevenir ataques de fuerza bruta.

R01F01T02P01 – Verificar que al cuarto intento consecutivo se devuelve error 429.

R01F02 – El administrador debe poder gestionar usuarios (crear, editar, desactivar).

R01F02T01 – Implementar endpoints CRUD en /users con control de acceso ADMIN.

R01F02T01P01 – Crear un usuario nuevo y verificar que aparece en el listado.

R01F02T01P02 – Verificar que un usuario con rol AGENT no puede acceder a /users.

R01F03 – El sistema debe soportar tres roles: ADMIN, SUPERVISOR y AGENT.

R01F03T01 – Implementar middleware requireRole para verificar permisos por rol en cada endpoint.

R01F03T01P01 – Verificar que cada rol accede solo a las funcionalidades permitidas.

R02 – El sistema debe gestionar conversaciones de WhatsApp

R02F01 – Visualizar un listado de conversaciones con filtros por estado y alcance.

R02F01T01 – Implementar GET /conversations con paginación, filtros de scope (mine/unassigned/all) y status (OPEN/PENDING/CLOSED).

R02F01T01P01 – Verificar que un agente solo ve sus conversaciones asignadas y las no asignadas.

R02F02 – Permitir la asignación y desasignación de conversaciones entre agentes.

R02F02T01 – Implementar endpoints assign-to-me, assign y unassign con lógica RBAC.

R02F02T01P01 – Verificar que un agente puede autoasignarse una conversación no asignada.

R02F02T01P02 – Verificar que un supervisor puede asignar cualquier conversación a cualquier agente activo.

R02F03 – Permitir cambiar el estado de las conversaciones.

R02F03T01 – Implementar PATCH /conversations/:id/status.

R02F03T01P01 – Cambiar estado de OPEN a PENDING y verificar la actualización en tiempo real.

R02F04 – Permitir editar el nombre de contacto de las conversaciones.

R02F04T01 – Implementar PATCH /conversations/:id/contact.

R03 – El sistema debe permitir el envío y recepción de mensajes en tiempo real

R03F01 – Visualizar el historial de mensajes de cada conversación con paginación por cursor.

R03F01T01 – Implementar GET /conversations/:id/messages con cursor-based pagination.

R03F01T01P01 – Cargar mensajes iniciales y verificar la carga de mensajes anteriores al hacer scroll.

R03F02 – Enviar mensajes a través del proveedor externo de mensajería.

R03F02T01 – Implementar POST /conversations/:id/messages con integración al API del proveedor y limitar el número de envíos por agente para evitar abusos.

R03F02T01P01 – Enviar un mensaje y verificar que se refleja en la conversación.

R03F02T01P02 – Verificar que al superar el límite de envíos por minuto se recibe un error.

R03F03 – Recibir mensajes y actualizaciones de estado en tiempo real via WebSockets.

R03F03T01 – Implementar Socket.IO con autenticación JWT y sistema de salas por conversación y rol.

R03F03T01P01 – Verificar que un mensaje entrante aparece instantáneamente sin recargar la página.

R04 – El sistema debe recibir eventos externos mediante webhooks

R04F01 – Procesar webhooks del proveedor de mensajería de forma segura.

R04F01T01 – Implementar POST /webhooks/provider con verificación de cabecera secreta (X-Webhook-Secret) y procesamiento asíncrono.

R04F01T01P01 – Enviar un webhook simulado y verificar la creación del mensaje en la base de datos.

R05 – El sistema debe ofrecer un panel de estadísticas

R05F01 – Mostrar KPIs y gráficos de rendimiento para supervisores y administradores.

R05F01T01 – Implementar GET /stats con filtros de fecha y métricas por agente.

R05F01T01P01 – Verificar que los KPIs reflejan los datos reales del sistema.

R05F02 – Permitir drill-down a las conversaciones de cada agente.

R05F02T01 – Implementar GET /stats/users/:userId/conversations.

R05F03 – El dashboard debe mostrar el número de conversaciones que tiene cada etiqueta.

R05F03T01 – Implementar GET /stats/labels para obtener el número de conversaciones por etiqueta. Acepta un parámetro opcional para filtrar por estado.

R05F03T01P01 – Verificar que los conteos coinciden con los datos reales de la base de datos.

R06 – La interfaz debe ser responsiva y adaptarse a múltiples dispositivos

R06F01 – Diseñar layout adaptativo para escritorio, tableta y móvil.

R06F01T01 – Implementar breakpoints responsive (desktop ≥ 1024 px, tablet 768-1023px, móvil < 768 px).

R06F01T01P01 – Verificar la navegación en cada tamaño de pantalla.

R07 – El sistema debe permitir clasificar conversaciones mediante etiquetas

R07F01 – El administrador debe poder gestionar etiquetas (crear, editar, desactivar).

R07F01T01 – Implementar endpoints CRUD en /labels con control de acceso ADMIN. Color almacenado como HEX y validado con regex. Borrado lógico con CASCADE sobre ConversationLabel.

R07F01T01P01 – Crear una etiqueta y verificar que aparece en el listado de conversaciones.

R07F02 – Los agentes deben poder asignar y quitar etiquetas en conversaciones (máximo 5).

R07F02T01 – Implementar POST /conversations/:id/labels y DELETE /conversations/:id/labels/:labelId. Validar límite de 5 etiquetas por conversación en el backend.

R07F02T01P01 – Asignar 5 etiquetas a una conversación y verificar que la sexta es rechazada con error 400.

R08 – El sistema debe analizar el sentimiento de los mensajes entrantes

R08F01 – El sistema debe calcular automáticamente el sentimiento de cada mensaje entrante y almacenarlo junto al mensaje.

R08F01T01 – Implementar sentiment.service.js con diccionario propio en español (~60 frases multipalabra + ~100 palabras). Escala de puntuación -5 a +5 con

categorías: angry (≤ -5), negative (-4 a -2), neutral (-1 a $+1$), positive ($\geq +2$).

Almacenar en campo sentiment de la tabla Message.

R08F01T01P01 – Enviar mensaje con lenguaje claramente negativo y verificar que el campo sentiment es "negative" o "angry" en la base de datos.

R09 – El sistema debe ofrecer un asistente de datos con inteligencia artificial

R09F01 – Supervisores y administradores deben poder realizar consultas en lenguaje natural sobre los datos del sistema.

R09F01T01 – Implementar POST /assistant con integración a RunPod (LLM) para generación de SQL a partir de lenguaje natural. Validar que solo se permiten consultas SELECT. Ejecutar en MySQL mediante Prisma y devolver respuesta resumida en lenguaje natural. Rate limit: 20 peticiones/minuto. Acceso restringido a SUPERVISOR y ADMIN.

R09F01T01P01 – Preguntar "¿cuántas conversaciones hay abiertas?" y verificar que la respuesta coincide con los datos reales de la base de datos.

R09F02 – La interfaz del asistente debe mantener historial de conversación y ofrecer sugerencias de preguntas.

R09F02T01 – Implementar AssistantPanel.vue en el dashboard con historial de turnos, chips de sugerencias e indicador de estado cold-start del endpoint de RunPod.

R09F02T01P01 – Realizar dos preguntas de seguimiento y verificar que el asistente mantiene el contexto de la conversación anterior.

R10 – El sistema debe gestionar el estado de lectura de las conversaciones por usuario

R10F01 – El sistema debe calcular el número de mensajes no leídos por cada conversación de forma individual para cada usuario.

R10F01T01 – Implementar la tabla ConversationUserState con la pareja (conversationId, userId) y un campo lastReadAt. En el listado de conversaciones se calcula cuántos mensajes son posteriores a esa fecha.

R10F01T01P01 – Verificar que el contador de no leídos disminuye al abrir la conversación.

R10F02 – Permitir marcar una conversación como leída.

R10F02T01 – Implementar POST /conversations/:id/read que actualiza lastReadAt para el usuario autenticado.

R10F02T01P01 – Verificar que tras llamar al endpoint el contador de mensajes no leídos pasa a cero.

DESCRIPCIÓN

En esta sección se describen los diagramas y la arquitectura de la solución propuesta para PowerChat.

Arquitectura de la solución

PowerChat sigue una arquitectura cliente-servidor basada en tres capas principales:

La capa de presentación (frontend) es una Single Page Application (SPA) desarrollada con Vue 3 y desplegada como contenido estático. Se comunica con el backend a través de peticiones HTTP REST y conexiones WebSocket persistentes mediante Socket.IO.

La capa de lógica de negocio (backend) es un servidor Node.js con Express que expone una API REST y gestiona las conexiones WebSocket. Se encarga de la autenticación (JWT), la autorización por roles (RBAC), el procesamiento de webhooks entrantes del proveedor de mensajería, y la difusión de eventos en tiempo real a los clientes conectados.

La capa de datos es una base de datos MySQL gestionada a través de Prisma ORM, que almacena usuarios, conversaciones, mensajes y estados de lectura.

Para la comunicación en tiempo real, el backend agrupa las conexiones de los clientes en distintas “salas” según el usuario, su rol y la conversación que tenga abierta. De este modo, cuando ocurre algo en una conversación (mensaje nuevo, cambio de estado o asignación), el servidor solo envía el evento a las salas que deberían recibirlo, sin difundir a todos los clientes conectados.

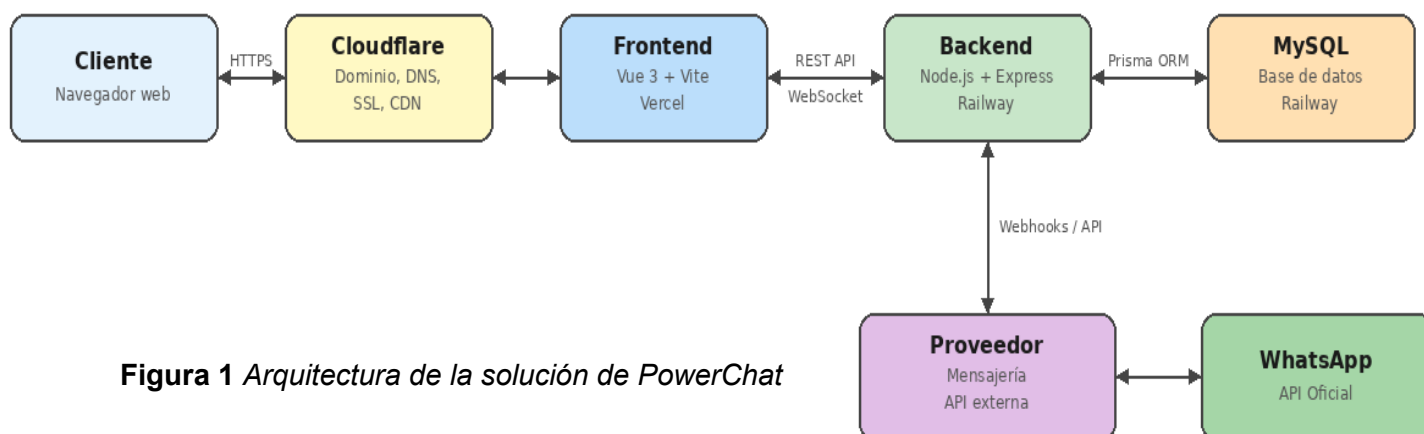


Figura 1 Arquitectura de la solución de PowerChat

Casos de uso

El sistema contempla tres actores principales (Agente, Supervisor/Administrador y Sistema/Proveedor externo) con los siguientes casos de uso principales:

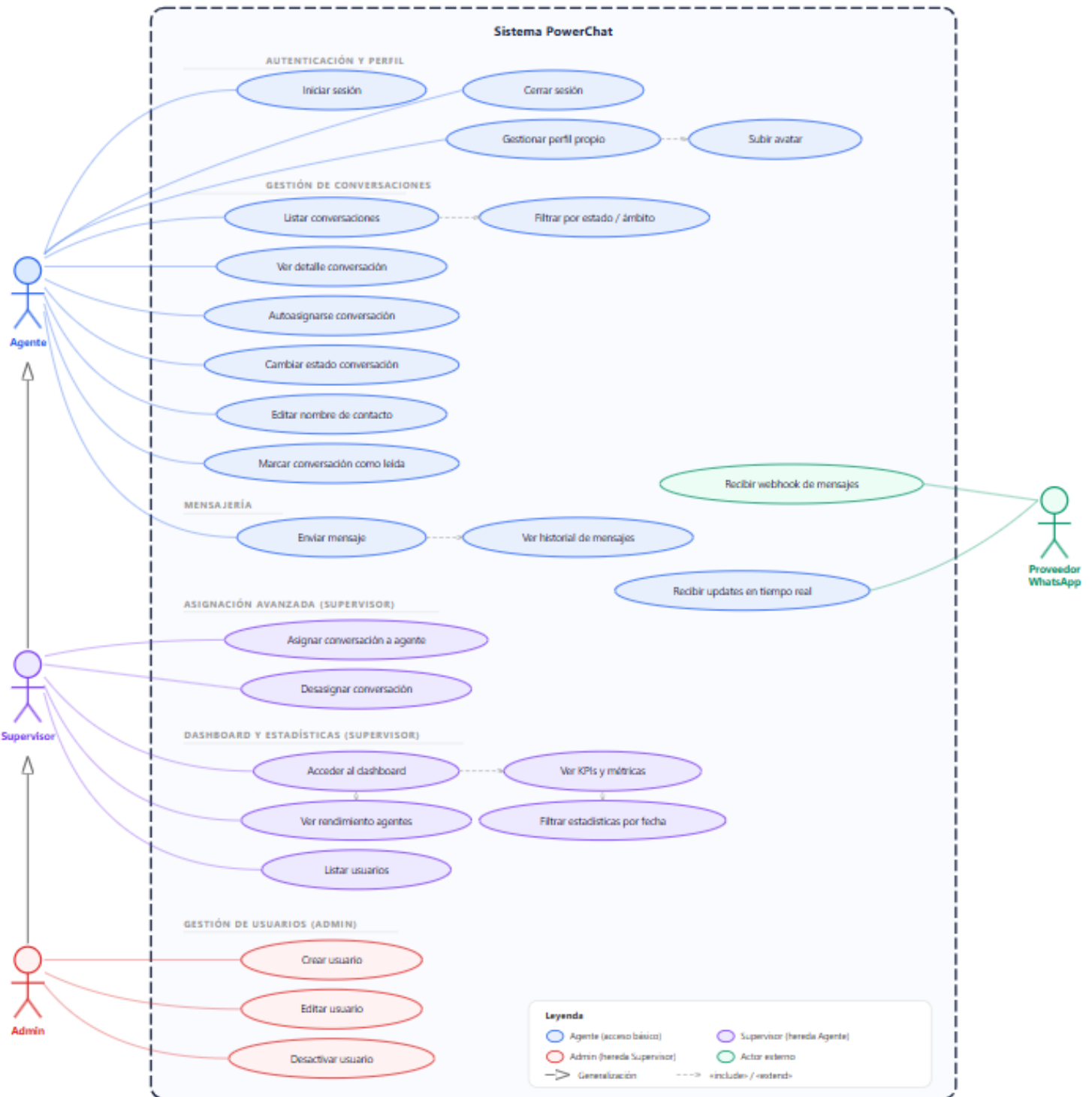


Figura 2: Diagrama de casos de uso UML

A continuación, se detallan los casos de uso más representativos:

Caso de uso: Gestionar conversaciones	
DESCRIPCIÓN: El agente visualiza, filtra y gestiona las conversaciones de WhatsApp asignadas o no asignadas, pudiendo auto asignarse conversaciones, cambiar su estado y enviar mensajes.	
PRECONDICIONES: Usuario autenticado con rol mínimo AGENT	POSTCONDICIONES: - Conversación asignada al agente - Estado actualizado - Mensaje enviado
DATOS ENTRADA: - Id conversación - Texto del mensaje - Nuevo estado	DATOS SALIDA: - Conversación actualizada - Mensaje creado con estado - Evento WebSocket emitido
TABLAS: - Conversation - Message - ConversationUserState	INTERFACES: - ConversationList.vue - MessageThread.vue - MessageInput.vue - ConversationHeader.vue

Tabla 1 Caso de uso: Gestionar conversaciones

Caso de uso: Consultar estadísticas	
DESCRIPCIÓN: El supervisor o administrador consulta los KPIs del sistema, gráficos de actividad y rendimiento por agente, con filtros de rango de fechas.	
PRECONDICIONES: Usuario autenticado con rol SUPERVISOR o ADMIN	POSTCONDICIONES: - Datos estadísticos mostrados - Gráficos renderizados
DATOS ENTRADA: - Fecha inicio - Fecha fin - Id agente	DATOS SALIDA: - KPIs (abiertas, pendientes, resueltas) - Gráficos por día Métricas por agente
TABLAS: - Conversation - Message - User	INTERFACES: - DashboardView.vue - KpiCard.vue - AgentTable.vue

Tabla 2 Caso de uso: Consultar estadísticas

DISEÑOS

Diagrama Entidad-Relación

La base de datos de PowerChat se compone de seis tablas principales interrelacionadas:

La tabla User almacena la información de los agentes, supervisores y administradores del sistema. Incluye campos para email (único), contraseña cifrada con scrypt, rol (ADMIN, SUPERVISOR o AGENT), datos personales, avatar y estado de activación (borrado lógico).

La tabla Conversation representa cada conversación de WhatsApp. Contiene un identificador externo del proveedor de mensajería, el teléfono del cliente, un nombre de contacto editable, el estado actual (OPEN, PENDING o CLOSED), una relación opcional con el usuario asignado, y campos de seguimiento del último mensaje.

La tabla Message almacena cada mensaje individual con su identificador externo, dirección (IN para entrantes, OUT para salientes), estado de entrega (PENDING, SENT, RECEIVED, READ, ERROR, DELETED), contenido de texto, marcas temporales y relaciones con la conversación y el usuario que envió el mensaje (para mensajes salientes).

La tabla ConversationUserState gestiona el estado de lectura por usuario y conversación, almacenando la fecha del último mensaje leído por cada usuario en cada conversación. La tabla Label permite clasificar conversaciones mediante etiquetas personalizables, cada una con un nombre único y un color en formato hexadecimal. La tabla ConversationLabel es la tabla intermedia que implementa la relación muchos a muchos entre conversaciones y etiquetas, con restricción única por par (conversación, etiqueta) y borrado en cascada.

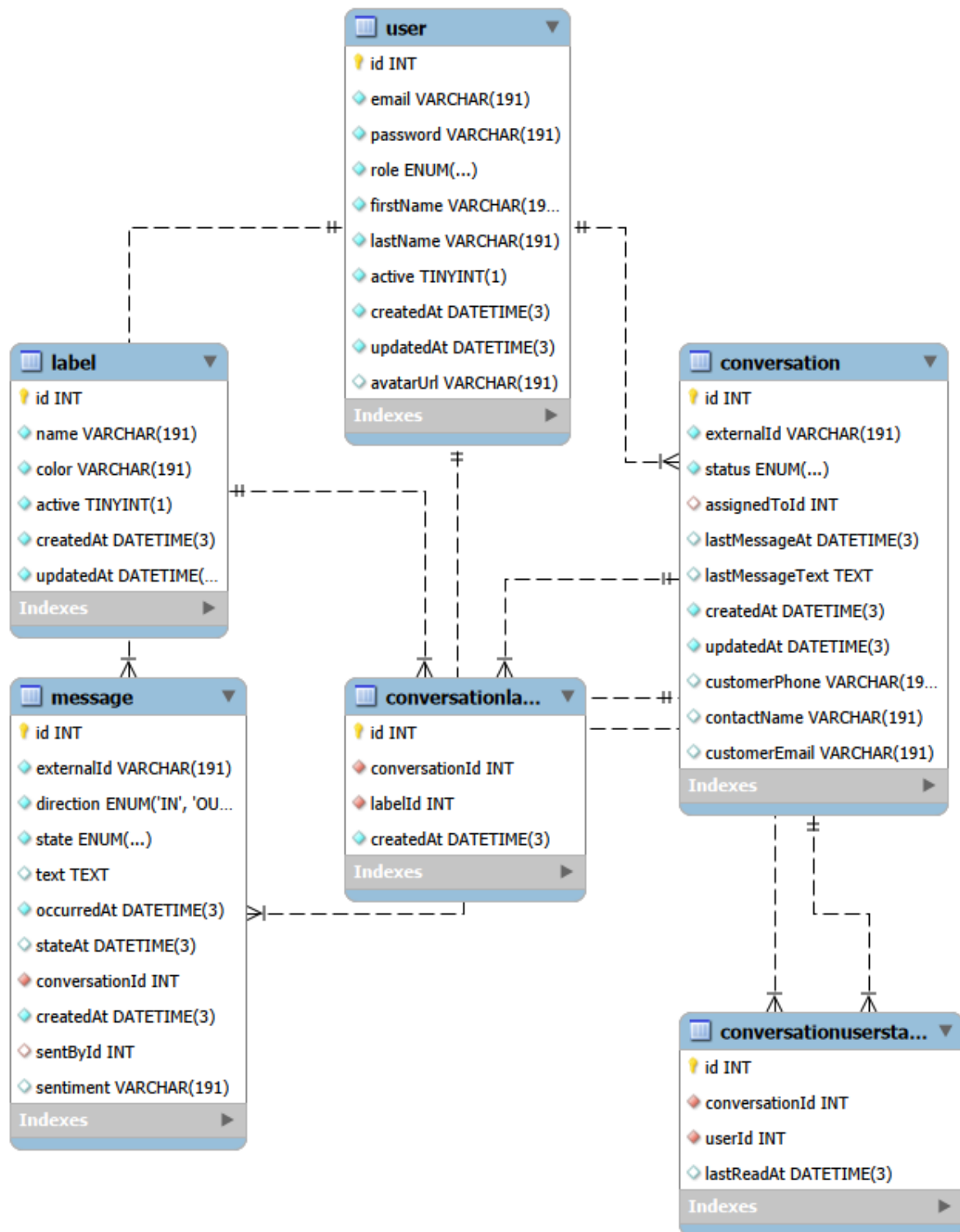


Figura 3. Diagrama E/R de la base de datos

Diagrama de la base de datos

A continuación, se muestra el esquema detallado de la base de datos con todos los campos, tipos de datos, índices y relaciones:

Tabla	Campo	Tipo	Descripción
User	id	Int (PK)	Identificador autoincremental
User	email	String (unique)	Correo electrónico único
User	password	String	Hash scrypt (salt:hash)
User	role	Enum	ADMIN, SUPERVISOR, AGENT
User	firstName, lastName	String	Nombre y apellidos
User	active	Boolean	Borrado lógico (default: true)
User	avatarUrl	String?	URL de la imagen de perfil

Conversation	id	Int (PK)	Identificador autoincremental
Conversation	externalId	String (unique)	ID del proveedor de mensajería
Conversation	customerPhone	String?	Teléfono del cliente
Conversation	contactName	String?	Nombre de contacto editable
Conversation	status	Enum	OPEN, PENDING, CLOSED
Conversation	assignedToId	Int? (FK)	Agente asignado
Conversation	lastMessageAt	DateTime?	Fecha último mensaje
Conversation	lastMessageText	Text?	Texto del último mensaje (preview en bandeja)

Message	id	Int (PK)	Identificador autoincremental
Message	externalId	String (unique)	ID del proveedor
Message	direction	Enum	IN (entrante), OUT (saliente)
Message	state	Enum	PENDING, SENT, RECEIVED, READ, ERROR, DELETED
Message	text	Text?	Contenido del mensaje
Message	conversationId	Int (FK)	Conversación asociada

Message	sentById	Int? (FK)	Agente que envió (mensajes OUT)
Message	occurredAt	DateTime	Timestamp del mensaje en el proveedor externo
Message	stateAt	DateTime?	Timestamp del último cambio de estado
Message	sentiment	String?	Sentimiento del mensaje: positive, neutral, negative, angry (solo mensajes IN)

ConvUserState	id	Int (PK)	Identificador autoincremental
ConvUserState	conversationId	Int (FK)	Conversación
ConvUserState	userId	Int (FK)	Usuario
ConvUserState	lastReadAt	DateTime?	Última lectura
Label	id	Int (PK)	Identificador autoincremental
Label	name	String (unique)	Nombre único de la etiqueta
Label	color	String	Color en formato HEX (#RRGGBB)
Label	active	Boolean	Borrado lógico (default: true)
ConversationLabel	id	Int (PK)	Identificador autoincremental
ConversationLabel	conversationId	Int (FK, cascade)	Conversación asociada (borrado en cascada)
ConversationLabel	labelId	Int (FK, cascade)	Etiqueta asociada. Unique(conversationId, labelId). Borrado en cascada

Tabla 3. Esquema detallado de la base de datos

Bandeja de conversaciones

La vista principal muestra un layout de dos columnas en escritorio: a la izquierda el listado de conversaciones con filtros (alcance y estado), y a la derecha el hilo de mensajes de la conversación seleccionada. En móvil se muestra como vista de panel único con navegación entre lista y chat.

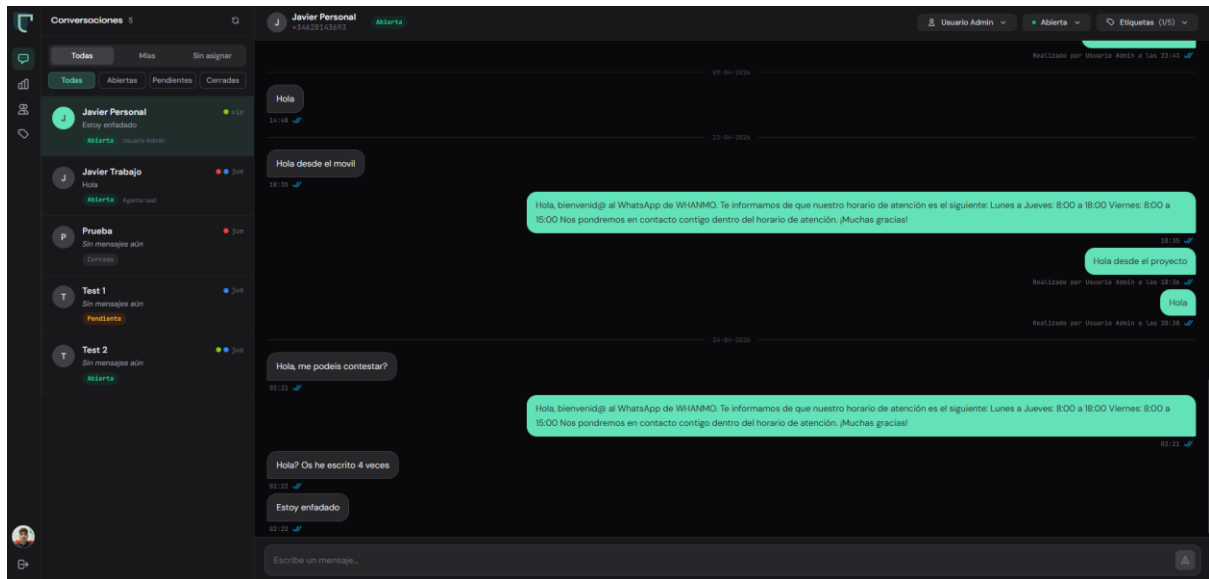


Figura 7. Pantalla principal en escritorio con chat abierto

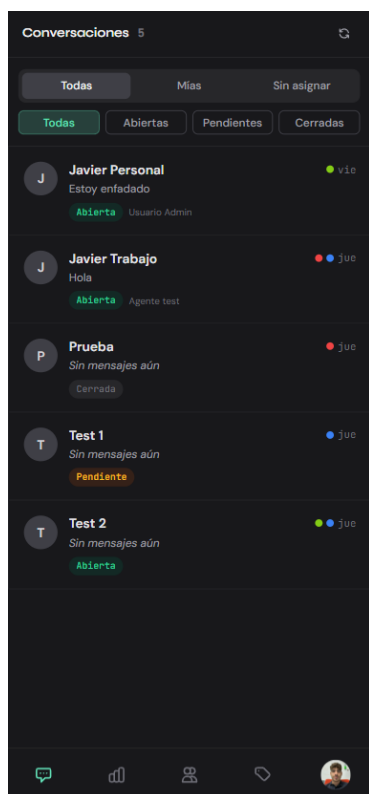


Figura 8. Listado de conversaciones en móvil

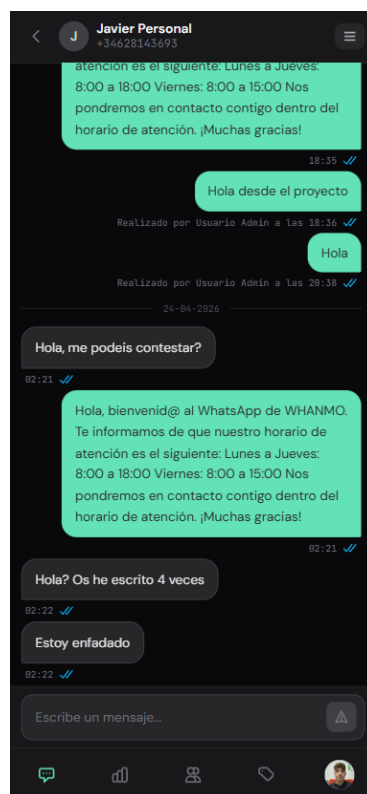


Figura 9. Chat abierto en móvil

Cabecera de conversación y asignación

La cabecera muestra el nombre del contacto (editable inline), teléfono, badge de estado, selector de agente para asignación, selector de estado y multiselector de etiquetas. En dispositivos pequeños, las acciones se agrupan en un menú hamburguesa.



Figura 10. Cabecera de conversación en escritorio

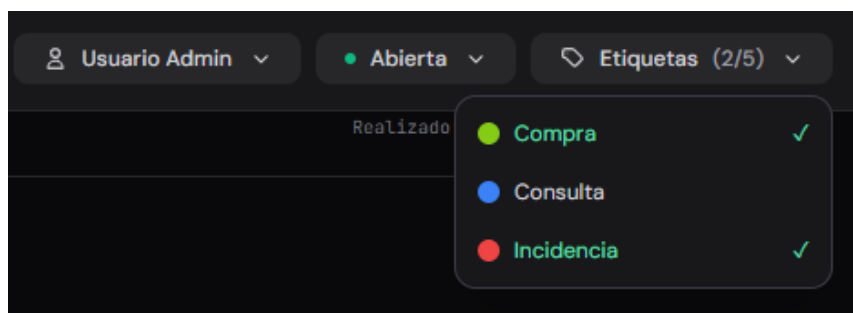


Figura 11. Desplegable multiselección de etiquetas en escritorio

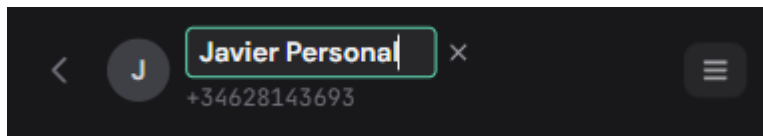


Figura 12. Nombre de contacto editable en móvil

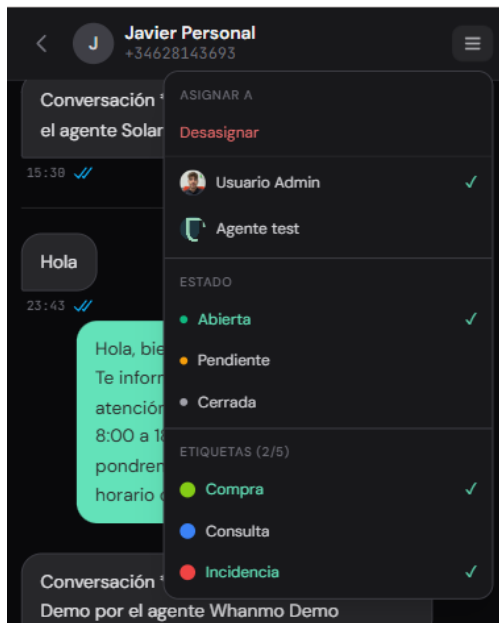


Figura 13. Desplegable de funcionalidades en móvil

Burbujas de mensaje

Los mensajes se muestran con estilo similar a WhatsApp: burbujas verdes para mensajes salientes y blancas para entrantes. Cada burbuja incluye el texto, la hora y un icono de estado de entrega (reloj, tick simple, doble tick gris, doble tick azul o error).

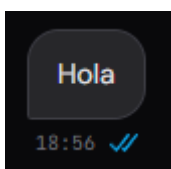


Figura 14. Mensaje entrante leído



Figura 15. *Mensaje entrante recibido*

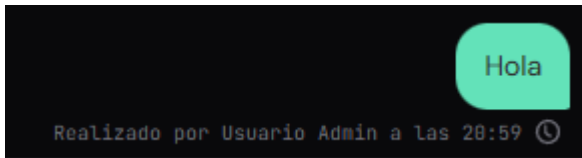


Figura 16. *Mensaje saliente enviando*

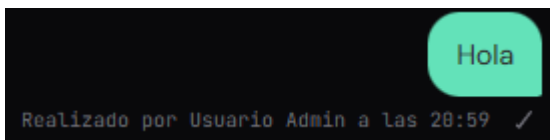


Figura 17. *Mensaje saliente enviado*

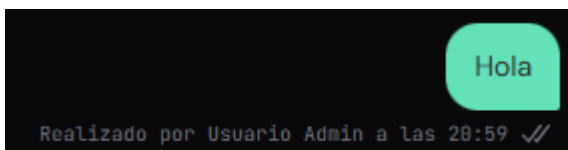


Figura 18. *Mensaje saliente recibido*

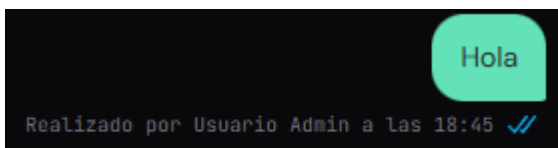


Figura 19. *Mensaje saliente leído*

Panel de estadísticas (Dashboard)

El dashboard muestra tarjetas KPI con métricas clave (conversaciones abiertas, sin asignar, asignadas, pendientes, resueltas), un gráfico de dona por estado, gráficos de línea de actividad diaria, y una tabla de rendimiento por agente con funcionalidad de drill-down.

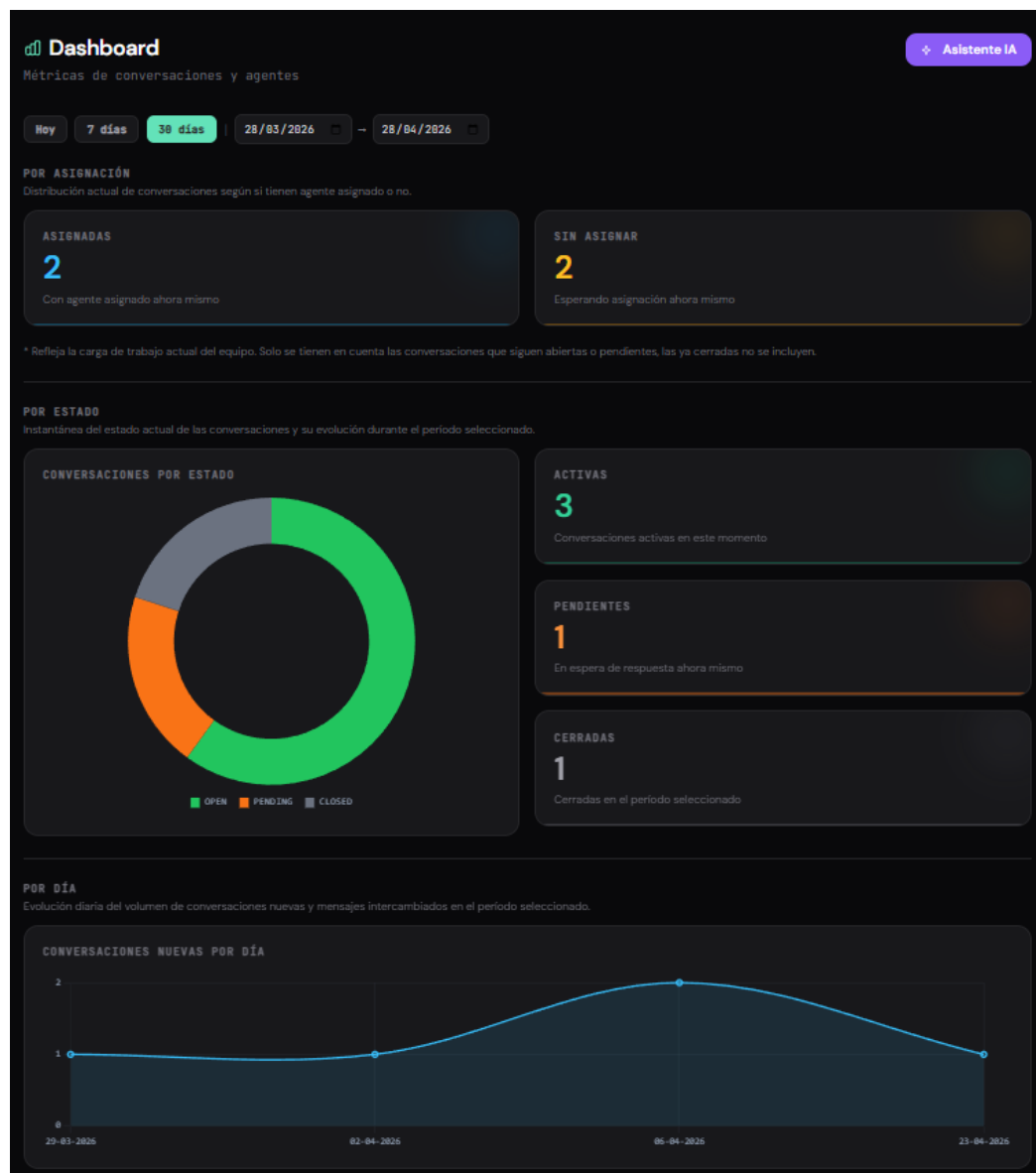


Figura 20. Captura parcial del dashboard en escritorio

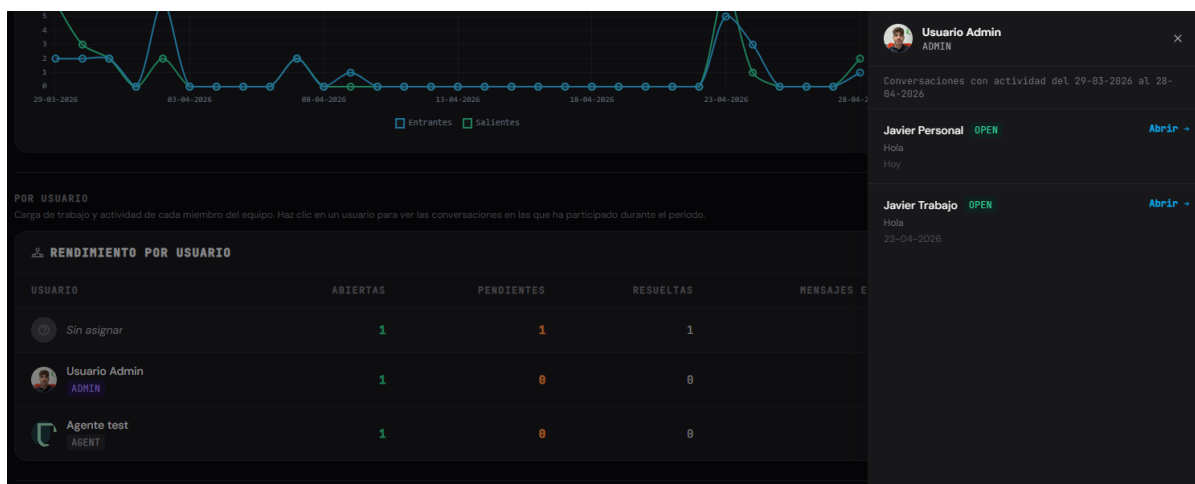


Figura 21. Drill-down por agente

Gestión de usuarios

La vista de gestión de usuarios (solo administradores) muestra una tabla con avatar, nombre, email, rol y estado de cada usuario. Incluye modales para crear y editar usuarios.

Gestión de usuarios + Nuevo usuario

2 usuarios registrados

USUARIO	EMAIL	ROL	ESTADO	ACCIONES
Usuario Admin	user@gmail.com	ADMIN	Activo	✎ Editar 🚫 Desactivar
Agente test	agente@gmail.com	AGENT	Activo	✎ Editar 🚫 Desactivar

Figura 22. Tabla de usuarios activos

Nuevo usuario ✕

NOMBRE APELLIDO

CORREO ELECTRÓNICO

CONTRASEÑA

ROL Selecciona un rol

Cancelar Crear usuario

Figura 23. Modal para crear usuario

Perfil de usuario

La vista de perfil permite ver la información del usuario autenticado, subir un avatar (arrastrar y soltar o clic), y alternar entre modo claro y oscuro.

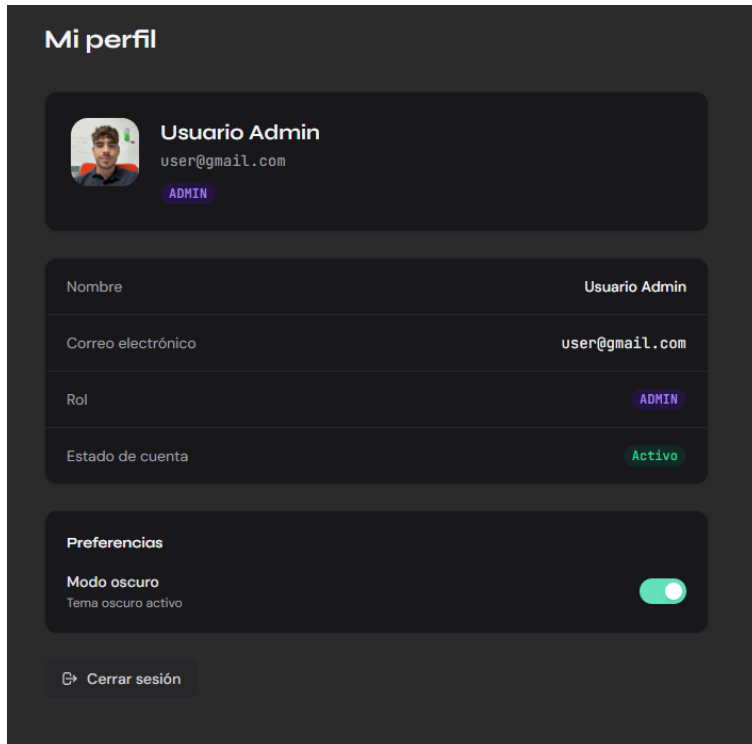


Figura 24. Perfil de usuario (modo oscuro)

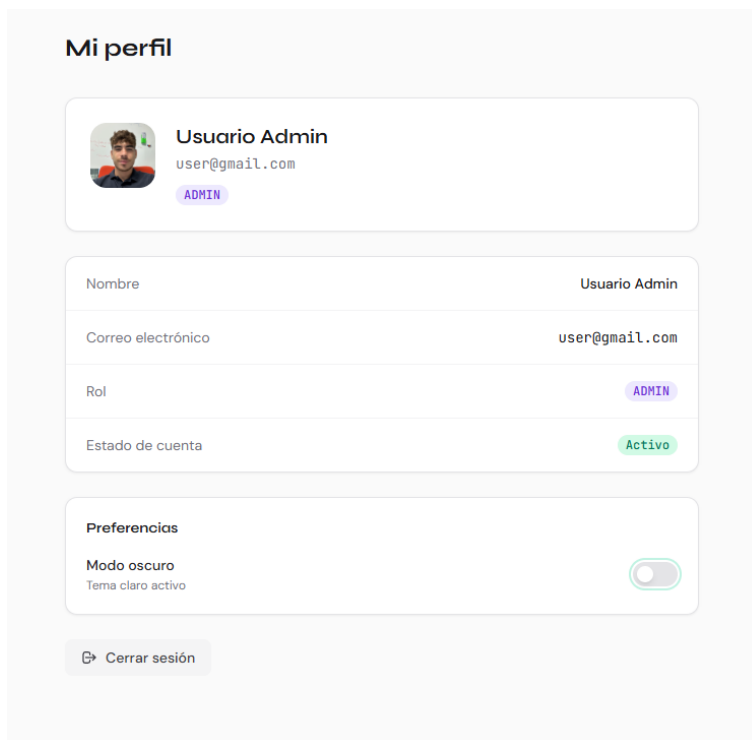
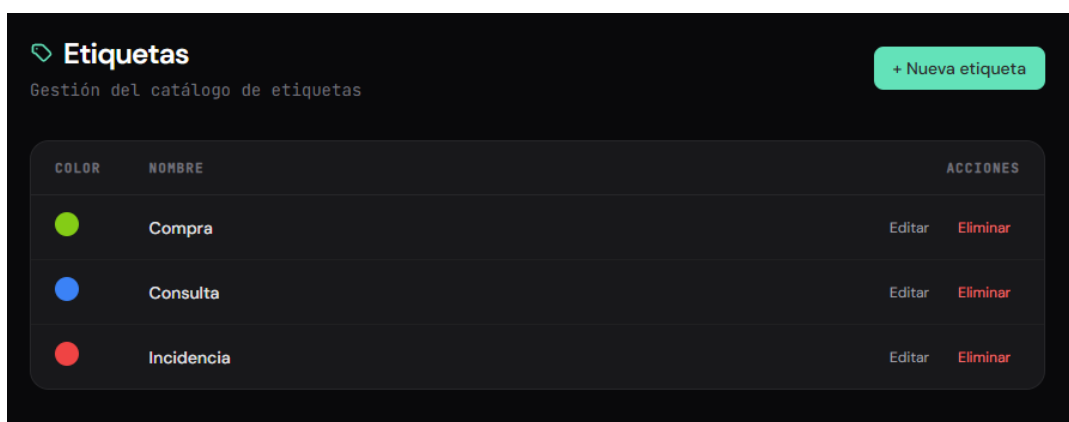


Figura 25. Perfil de usuario (modo claro)

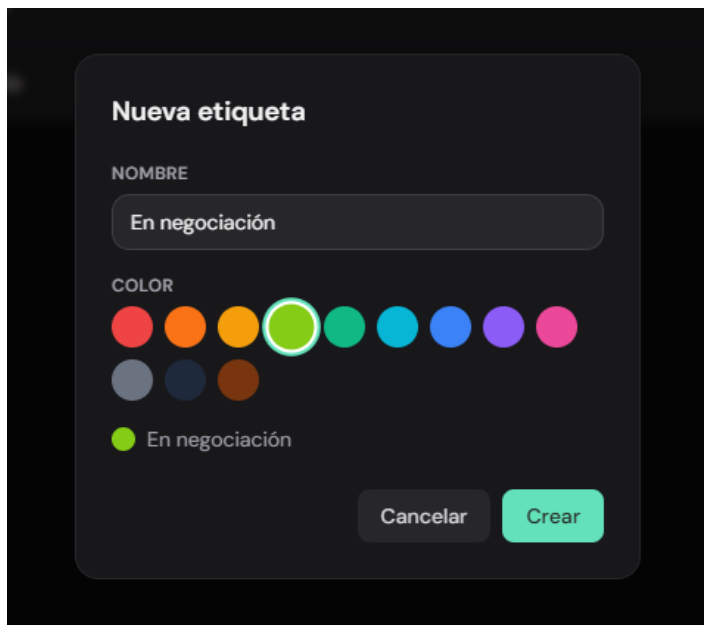
Gestión de etiquetas

La vista de gestión de etiquetas (solo administradores) permite crear, editar y eliminar etiquetas personalizables. Cada etiqueta tiene un nombre único y un color seleccionable de una paleta de 12 tonos en formato hexadecimal, con previsualización en tiempo real. La vista muestra una tabla con todas las etiquetas activas y un botón para crear nuevas. El modal de creación y edición incluye el campo de nombre y el selector de color con muestra del resultado.



COLOR	NOMBRE	ACCIONES
	Compra	Editar Eliminar
	Consulta	Editar Eliminar
	Incidencia	Editar Eliminar

Figura 26. Tabla de etiquetas existentes



Nueva etiqueta

NOMBRE

En negociación

COLOR

 En negociación

Cancelar Crear

Figura 27. Modal de creación de etiquetas

Las etiquetas son visibles en la bandeja de conversaciones como puntos de color en cada tarjeta, y en la cabecera de la conversación como un desplegable multiselección que permite asignar o quitar etiquetas con un máximo de cinco por conversación.

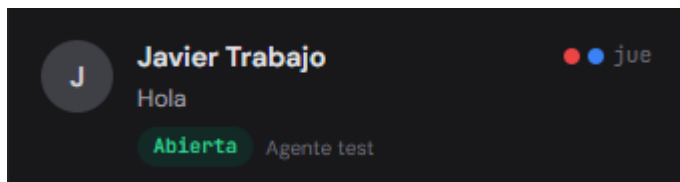


Figura 28. *Puntos de etiquetas en tarjeta de conversación*

Asistente de datos (IA)

El panel del asistente de datos está integrado en el dashboard y es accesible para supervisores y administradores. Permite realizar preguntas en lenguaje natural sobre los datos del sistema (conversaciones, mensajes, agentes, etiquetas) y recibir respuestas en lenguaje natural generadas a partir de consultas SQL ejecutadas en tiempo real sobre la base de datos.

La interfaz muestra un historial de conversación con el asistente, chips de sugerencias de preguntas frecuentes para facilitar el primer uso, un campo de texto para escribir la pregunta y un indicador de estado cuando el modelo está arrancando (cold start). Las preguntas de seguimiento mantienen el contexto de la conversación anterior.

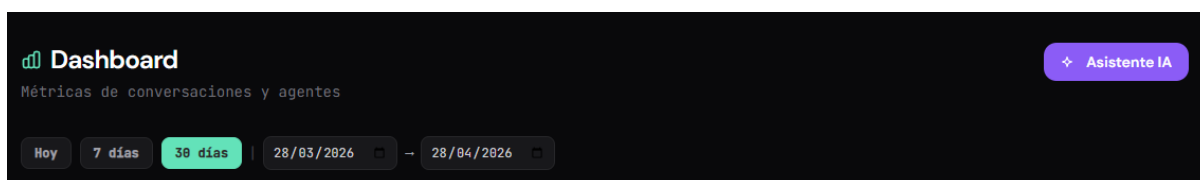


Figura 29. *Panel del asistente de datos en el dashboard*

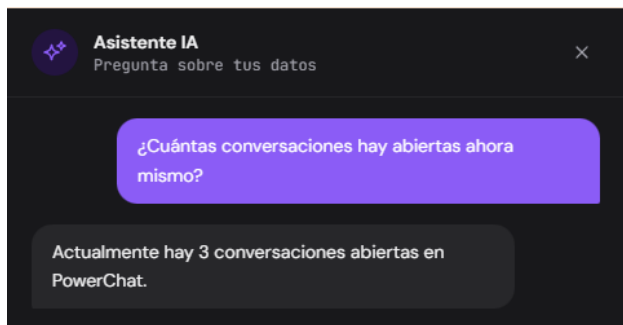


Figura 30. *Ejemplo de pregunta y respuesta del asistente*

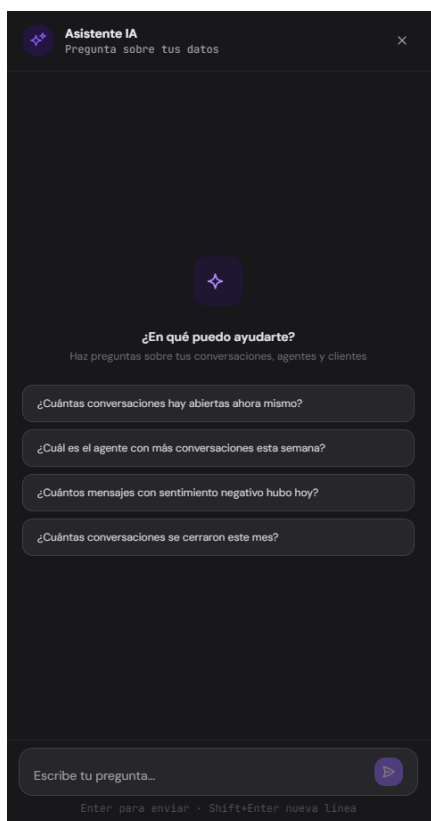


Figura 31. *Chips de sugerencias de respuestas*

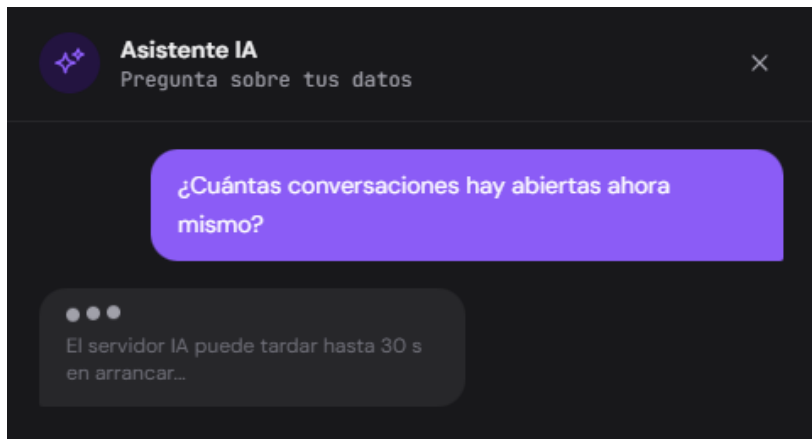


Figura 32. Indicador cold-start del asistente de IA

TECNOLOGÍA

En este apartado se recogen las tecnologías que he utilizado para desarrollar PowerChat, explicando para qué sirve cada una y por qué la elegí.

Backend

Node.js: es el entorno que permite ejecutar JavaScript fuera del navegador, directamente en el servidor (OpenJS Foundation, s. f.). Lo elegí porque me permite usar el mismo lenguaje (JavaScript) tanto en el backend como en el frontend, lo cual simplifica mucho el desarrollo. Además, funciona muy bien para aplicaciones en tiempo real como PowerChat porque gestiona muchas conexiones simultáneas de forma eficiente.

Express v5: es el framework más popular de Node.js para crear APIs (StrongLoop/Express, s. f.). Facilita mucho la definición de rutas (endpoints), la gestión de peticiones HTTP y la aplicación de middlewares como la autenticación o el rate limiting. En PowerChat lo uso para montar toda la API REST del proyecto (28 endpoints en total).

Prisma ORM: es una herramienta que facilita trabajar con bases de datos desde Node.js (Prisma, s. f.). En lugar de escribir consultas SQL a mano, defines el esquema de tus tablas en un fichero y Prisma genera automáticamente las funciones para hacer consultas,

inserciones, actualizaciones, etc. También gestiona las migraciones de la base de datos. Lo uso para todas las operaciones con MySQL.

Socket.IO v4: es la librería que uso para la comunicación en tiempo real (Socket.IO, s. f.).

Funciona sobre WebSockets y permite que el servidor envíe datos al navegador instantáneamente sin que el cliente tenga que estar preguntando continuamente. En PowerChat lo uso para que los mensajes nuevos, los cambios de estado y las asignaciones aparezcan al momento en la pantalla de todos los agentes conectados.

JSON Web Token (JWT): es el sistema que utilizo para la autenticación (Auth0, s. f.).

Cuando un usuario hace login, el servidor le genera un token que incluye su id y su rol. Ese token se envía en cada petición posterior para identificar al usuario sin necesidad de mantener sesiones en el servidor. Ya tenía experiencia previa trabajando con JWT en mi entorno profesional, lo que me permitió implementarlo de forma rápida y segura.

MySQL: es la base de datos relacional (Oracle Corporation, s. f.) que almacena toda la información del sistema: usuarios, conversaciones, mensajes, etiquetas, estados de lectura y análisis de sentimientos. La elegí por ser una de las bases de datos más utilizadas, con gran estabilidad y buen rendimiento. Además, ya la conocía del ciclo formativo.

Multer: es la librería de Node.js que uso para gestionar la subida de archivos. En PowerChat la utilizo para el upload del avatar de perfil de los usuarios. Valida el tipo de archivo (JPG, PNG, WebP) y el tamaño máximo (2 MB) antes de guardarlo en el servidor.

RunPod: es la plataforma de inferencia de modelos de lenguaje (RunPod, s. f.) que utilizo para el asistente de datos con inteligencia artificial. RunPod permite desplegar modelos LLM open-source y exponerlos mediante una API REST. En PowerChat utilizo un modelo open-source especializado en generación de código (de la familia qwen2.5-coder), servido a través de Ollama. Lo uso para dos tareas: primero, convertir la pregunta del usuario en una consulta SQL válida (Text-to-SQL); y segundo, resumir el resultado de la consulta en lenguaje natural para mostrárselo al usuario. Las llamadas se realizan al endpoint /runsync

con un timeout de 55 segundos.

A nivel de seguridad, las contraseñas de los usuarios se almacenan hasheadas con script (incluido en Node.js), las peticiones autenticadas viajan con JWT, los webhooks del proveedor se validan con un secreto compartido y los endpoints más sensibles (login, envío de mensajes y asistente IA) están protegidos con rate limiting para evitar abusos.

Frontend

Vue 3 (Composition API): es el framework de JavaScript que uso para construir toda la interfaz de usuario (Vue.js, s. f.). Con Vue puedes crear componentes reutilizables y la página se actualiza automáticamente cuando cambian los datos. Uso la Composition API con la sintaxis `<script setup>`, que es la forma más moderna y limpia de organizar el código en Vue.

Vite: es la herramienta que uso para arrancar el servidor de desarrollo y para generar el build de producción del frontend (Vite, s. f.). Es mucho más rápido que Webpack y los cambios que haces en el código se reflejan al instante en el navegador sin recargar la página.

Pinia: es el gestor de estado oficial de Vue 3 (Pinia, s. f.) y sustituye a Vuex. Lo uso para guardar datos que necesitan compartirse entre varios componentes, como la información del usuario logueado, la lista de conversaciones, las estadísticas del dashboard y el tema claro/oscuro.

Tailwind CSS: es un framework de CSS que funciona con clases de utilidad (Tailwind Labs, s. f.). En vez de escribir hojas de estilo separadas, aplicas las clases directamente en el HTML. Esto me ha permitido maquetar toda la interfaz de forma rápida, incluyendo el modo oscuro y la adaptación a diferentes tamaños de pantalla (móvil, tableta y escritorio).

Chart.js: es una librería para crear gráficos en la web (Chart.js, s. f.). La uso en el dashboard para pintar el gráfico de dona con el reparto de conversaciones por estado y los gráficos de líneas con la actividad diaria. La integro en Vue a través de vue-chartjs, que es el wrapper oficial que expone los componentes de Chart.js como componentes Vue reutilizables.

Herramientas de desarrollo

Git: lo uso para el control de versiones del código. Me permite llevar un historial de todos los cambios y volver atrás si algo falla.

Postman: herramienta que he usado para probar todos los endpoints de la API durante el desarrollo. Tengo colecciones organizadas por funcionalidad (auth, conversaciones, usuarios, etc.).

Visual Studio Code: el editor de código que he usado durante todo el proyecto, con extensiones para Vue, Prisma y Tailwind CSS.

Despliegue

Para poner la aplicación en producción y que sea accesible desde internet bajo el dominio propio testjgarcia.com, he utilizado cuatro servicios en la nube: Vercel para el frontend, Railway para el backend y la base de datos, RunPod para el modelo de lenguaje del asistente de datos, y Cloudflare para la gestión del dominio y DNS. A continuación se detalla la configuración de cada uno.

Frontend: Vercel

Vercel es la plataforma donde he desplegado el frontend de PowerChat (la SPA de Vue 3). Está especialmente optimizada para aplicaciones frontend modernas y ofrece integración

directa con repositorios de GitHub: cada vez que se sube un cambio a la rama principal, Vercel lanza automáticamente un nuevo build con Vite y lo publica en su CDN global, sirviendo los archivos estáticos desde el servidor más cercano al usuario.

La configuración del proyecto en Vercel requiere definir el directorio raíz del frontend, el comando de build (`npm run build`) y el directorio de salida (`dist`). Las variables de entorno del frontend, como la URL base de la API y la URL del servidor de WebSockets, se configuran directamente en el panel de Vercel para que estén disponibles durante el proceso de build.

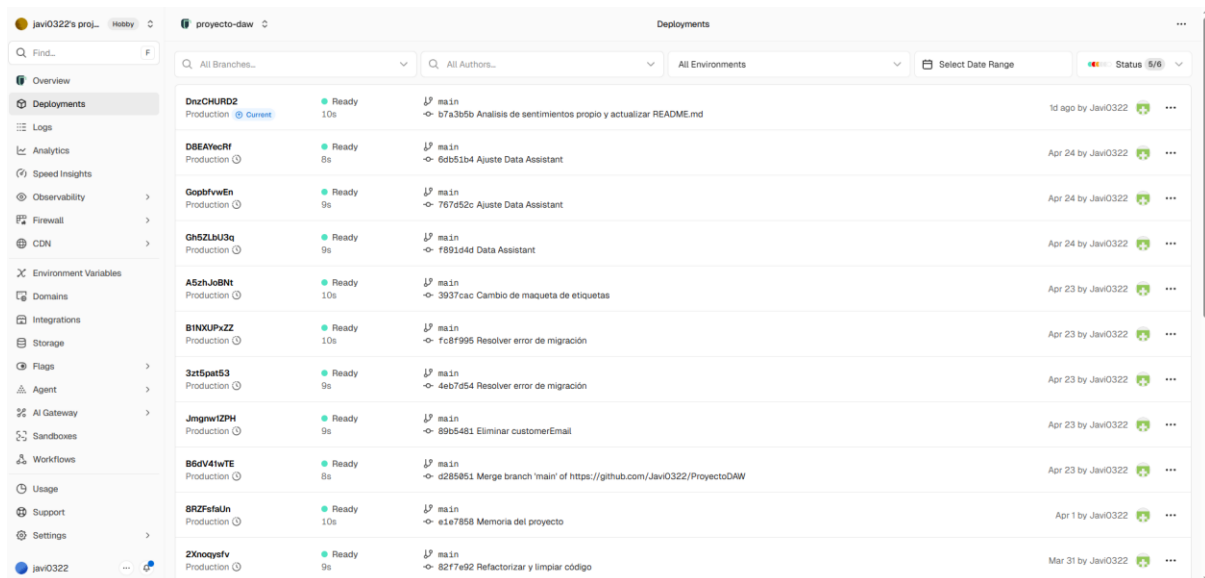


Figura 33. Panel de proyecto en Vercel con el historial de deployments

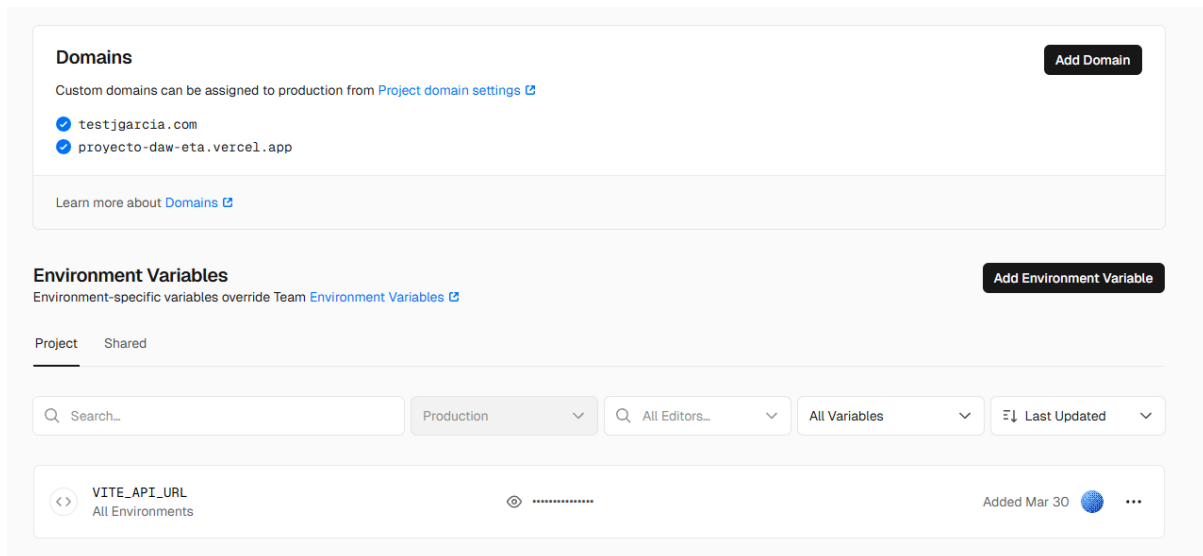


Figura 34. Sección de variables de entorno del proyecto en Vercel

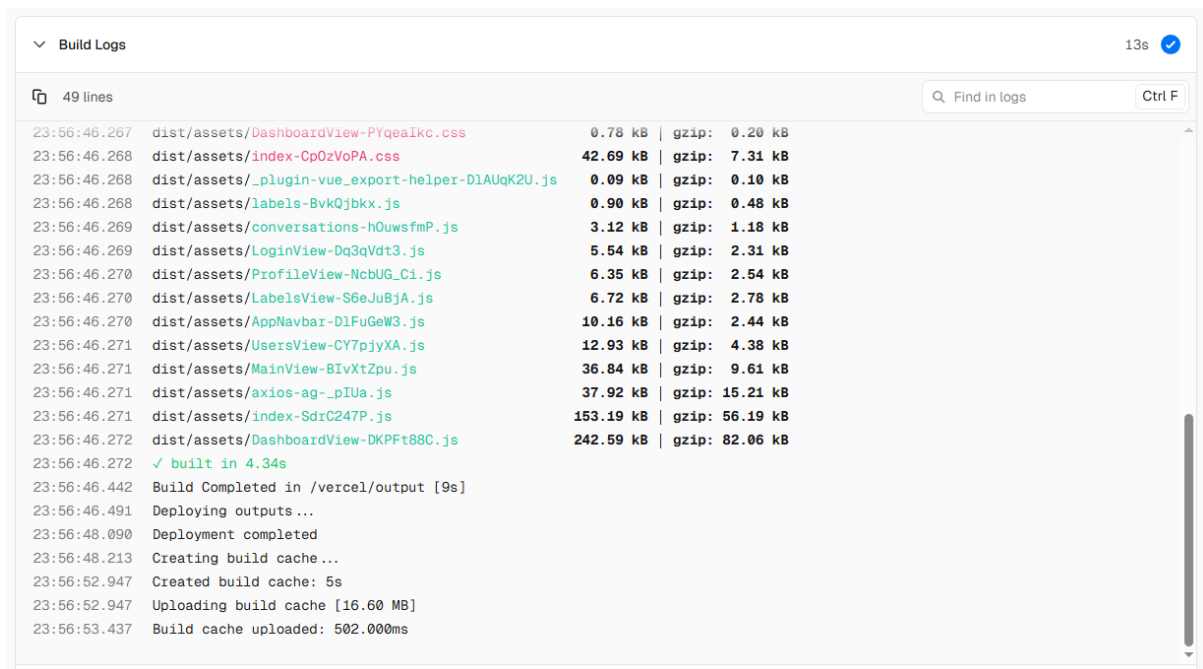


Figura 35. Log de build exitoso en Vercel

Backend y base de datos: Railway

Railway es la plataforma de despliegue en la nube donde alojan tanto el servidor Node.js con Express como la base de datos MySQL. Railway permite crear un proyecto con varios servicios vinculados entre sí: en este caso, un servicio de aplicación conectado al

repositorio de Git y un servicio de base de datos MySQL gestionado por la propia plataforma.

El servicio de backend se despliega automáticamente con cada push a la rama principal. Railway detecta que es una aplicación Node.js, instala las dependencias, ejecuta las migraciones de Prisma (prisma migrate deploy) y arranca el servidor. La URL pública del backend la genera Railway automáticamente con certificado SSL, y es la que se configura como CORS_ORIGIN y como destino de los webhooks del proveedor de mensajería.

El servicio de MySQL de Railway proporciona automáticamente la variable DATABASE_URL con la cadena de conexión completa, que Prisma utiliza para conectarse a la base de datos. Railway también permite visualizar las tablas y ejecutar consultas desde su panel, lo que ha resultado útil para verificar el estado de los datos durante el desarrollo.

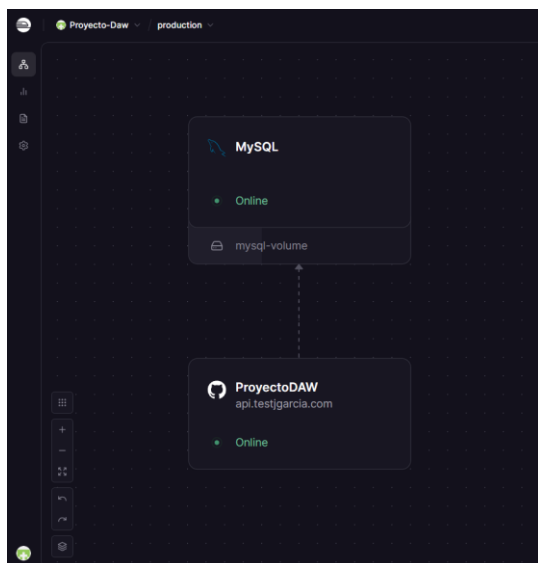


Figura 36. Panel de proyecto en Railway con los dos servicios

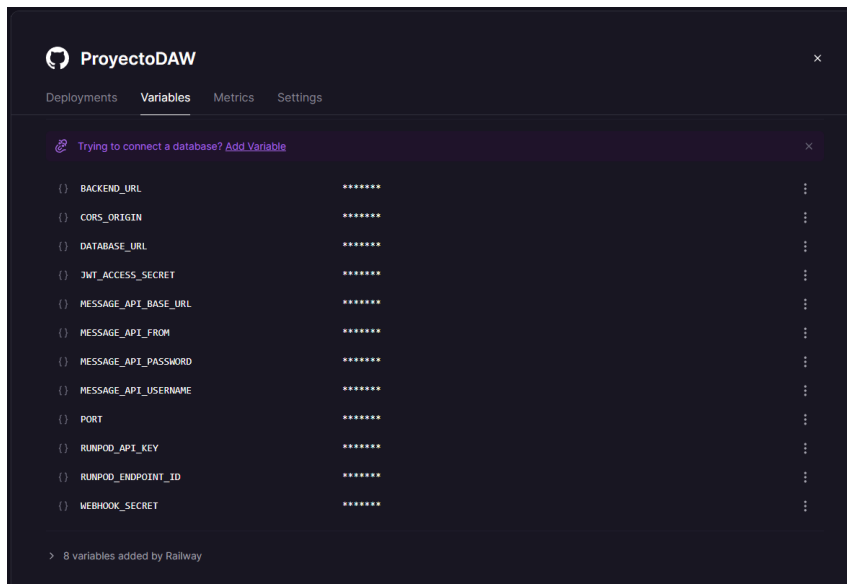


Figura 37. Variables de entorno del servicio backend en Railway

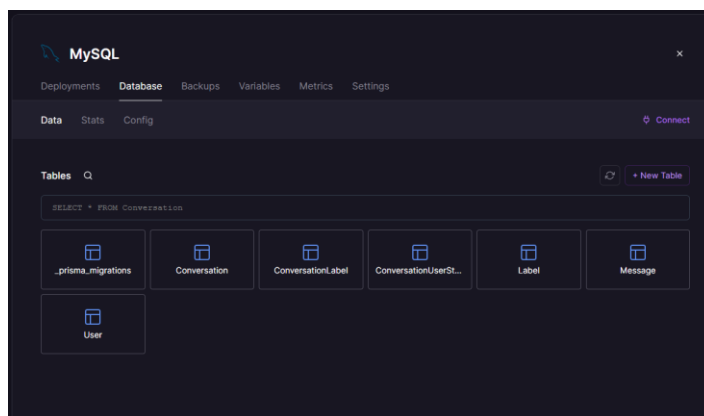


Figura 38. Panel del servicio MySQL en Railway con las tablas

Modelo de lenguaje: RunPod

RunPod es la plataforma de inferencia de modelos de lenguaje (LLM) que utilizo para alimentar el asistente de datos con inteligencia artificial. A diferencia de Railway y Vercel, donde el código se despliega desde un repositorio Git, en RunPod se selecciona un modelo de lenguaje preentrenado disponible en su catálogo y se configura como un endpoint serverless al que el backend realiza llamadas HTTP.

El endpoint de RunPod funciona de forma serverless: cuando no hay peticiones activas, el modelo se descarga de memoria para reducir costes. Cuando llega una petición, RunPod carga el modelo en una GPU y procesa la inferencia. Esta característica explica el indicador de cold start que se muestra en el AssistantPanel.vue: la primera pregunta puede tardar entre 30 y 55 segundos mientras el modelo arranca, y las siguientes son prácticamente instantáneas.

La integración con el backend se realiza a través de dos variables de entorno:

RUNPOD_API_KEY (clave de autenticación de la cuenta) y RUNPOD_ENDPOINT_ID (identificador del endpoint serverless desplegado).

The screenshot shows the RunPod dashboard for the endpoint 'Runpod Worker Ollama ollama@0.21.1'. The interface is in a light blue theme. On the left is a sidebar with various navigation links. The main content area shows the endpoint's status as 'Running' with a cost of '\$0.0003/s'. Below this, there's a table of workers. The table has columns for Worker ID, Location, GPU type, Uptime, and Version. The workers listed are:

Worker ID	Location	GPU type	Uptime	Version
see88ufdcv3j0a	US	RTX 5090	8.34s	2 Latest
er1gmughtjw3	IE	RTX 5090	-	2 Latest
gg2hurf042a1av	IE	RTX 5090	-	2 Latest
vu32348poyxtr1	IE	RTX 5090	-	2 Latest
bk35p2stogv6w	NO	RTX 5090	-	2 Latest

At the top of the main panel, there's a notification: 'A new version of this endpoint is available!' with buttons 'Skip this version' and 'Update endpoint'. The table also has a 'Filter by worker ID' dropdown and a '25 per page' selector at the bottom.

Figura 39. Panel de RunPod con el endpoint configurado

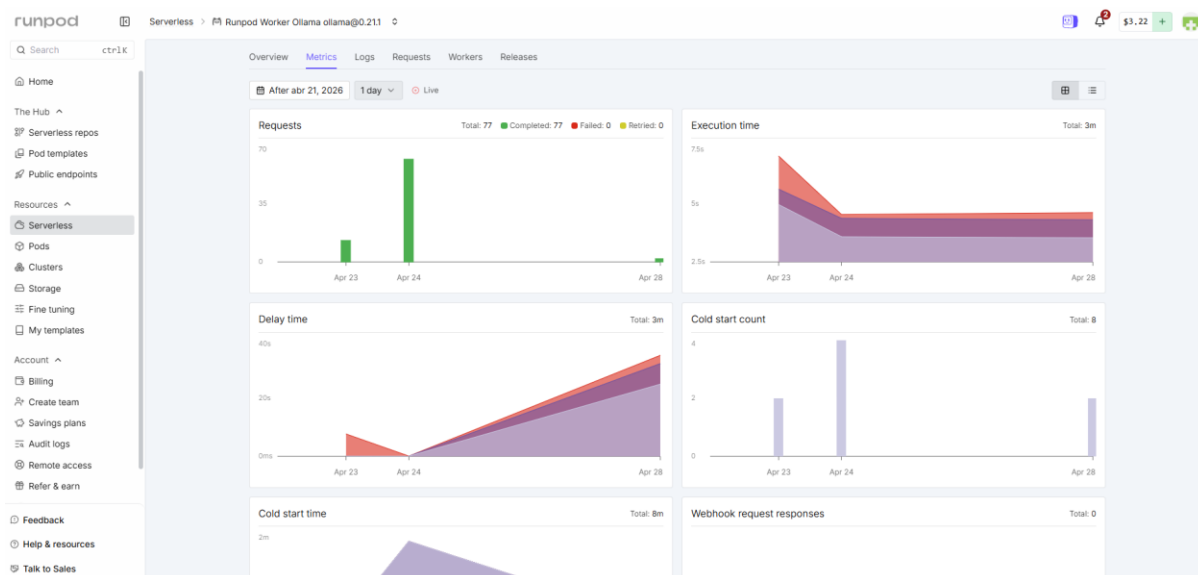


Figura 40. Métricas del endpoint en RunPod

DNS y seguridad: Cloudflare

Cloudflare actúa como intermediario entre el usuario y los servidores, gestionando el dominio personalizado `testjgarcia.com` y su configuración DNS. Se han configurado dos registros CNAME bajo ese dominio: `testjgarcia.com` apunta al frontend desplegado en Vercel, y `api.testjgarcia.com` apunta a la URL pública del backend en Railway. De este modo, todo el tráfico pasa primero por Cloudflare, que proporciona protección DDoS, caché de contenido estático y gestión automática de certificados SSL para ambos subdominios.

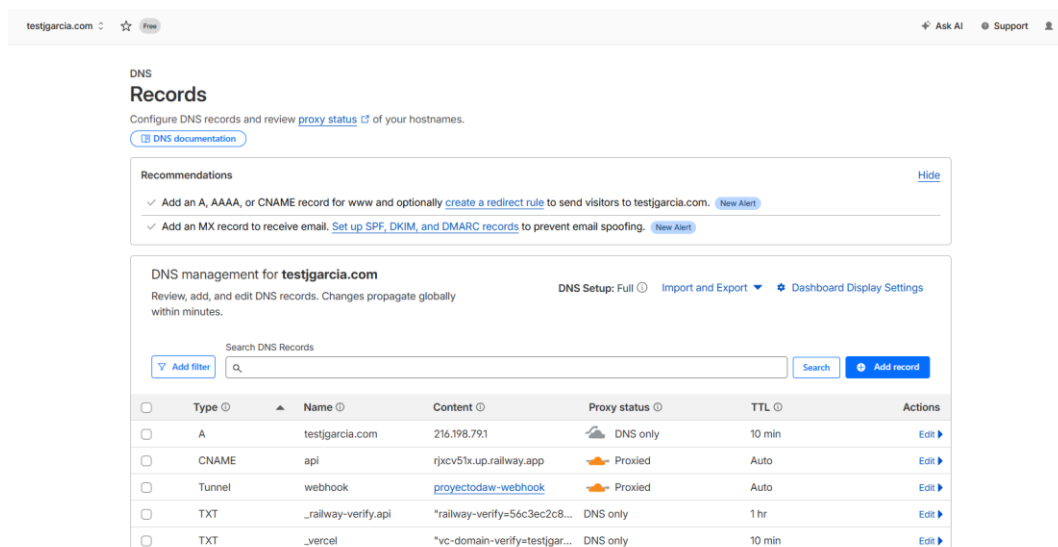


Figura 41. Panel de DNS en Cloudflare con los registros configurados

Arquitectura de despliegue completa

La arquitectura de despliegue queda así: el usuario accede a testjgarcia.com, gestionado por Cloudflare, que redirige las peticiones del frontend a Vercel y las peticiones de la API (api.testjgarcia.com) al backend en Railway, donde también se encuentra la base de datos MySQL. Cuando el usuario utiliza el asistente de datos, el backend de Railway realiza llamadas al endpoint serverless de RunPod para obtener la respuesta del modelo de lenguaje. El proveedor de mensajería externo envía los webhooks directamente a api.testjgarcia.com, que Cloudflare redirige al backend en Railway.

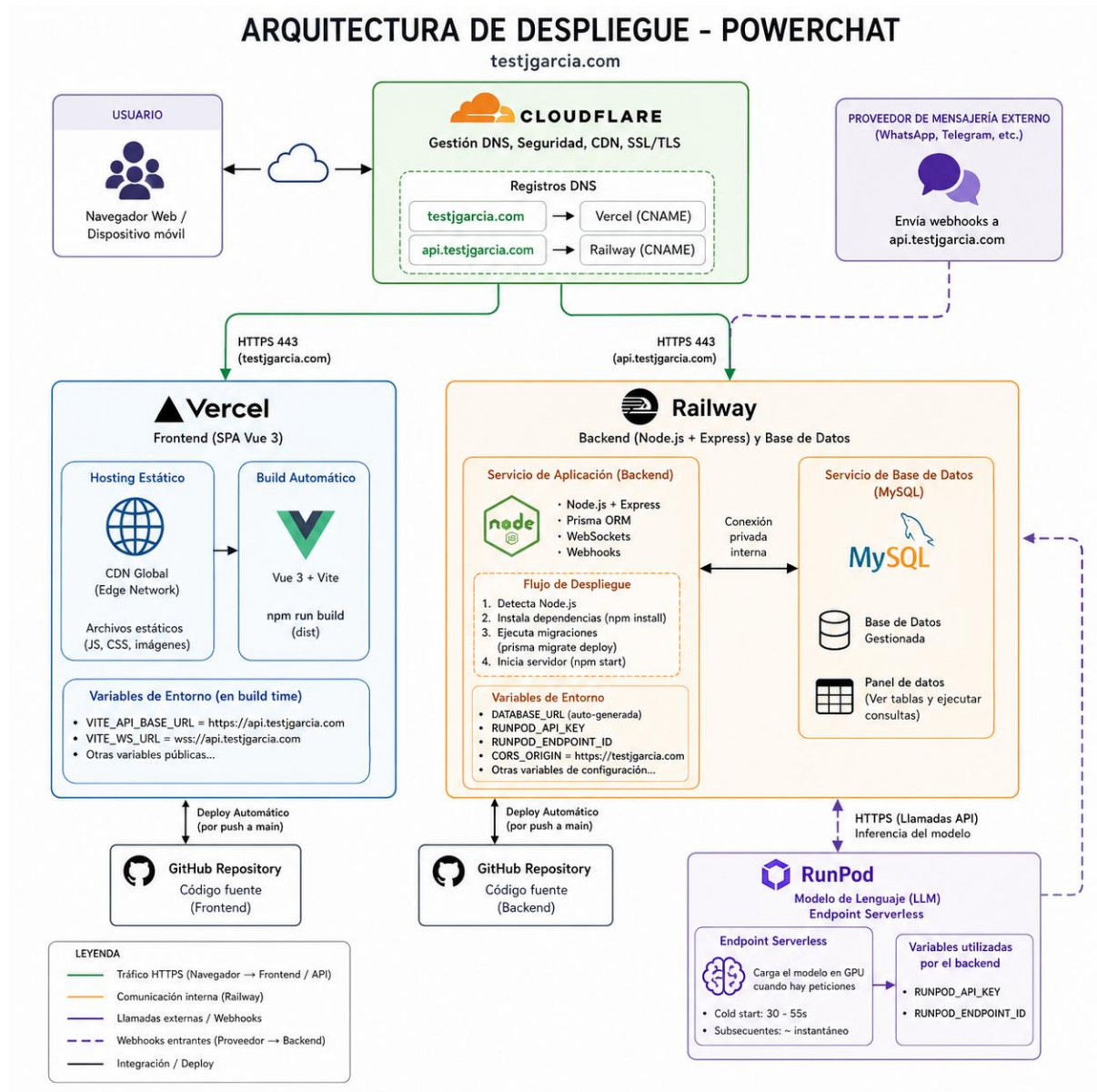


Figura 42. Diagrama de arquitectura de despliegue completo

METODOLOGÍA

Metodología utilizada: Desarrollo iterativo incremental

Para el desarrollo de PowerChat se ha seguido una metodología iterativa e incremental. Esta aproximación consiste en construir el sistema de forma progresiva, añadiendo funcionalidades completas una a una. En cada iteración se completa un ciclo de análisis, diseño, implementación y pruebas para una funcionalidad concreta, que se integra al sistema antes de abordar la siguiente.

Se ha elegido esta metodología por las siguientes razones: el proyecto ha sido desarrollado por un único programador, lo que hace innecesario el overhead de marcos más formales como Scrum; permite obtener un producto funcional desde las primeras iteraciones; facilita la detección temprana de problemas al probar cada funcionalidad de forma aislada; y ofrece flexibilidad para reajustar el alcance según el tiempo disponible.

Las iteraciones principales del proyecto han sido las siguientes, siempre completando primero el backend de cada funcionalidad y luego su frontend correspondiente:

Iteración 1: Configuración del entorno. Se estableció la estructura del proyecto (backend con Express y Prisma, frontend con Vue y Vite), se configuró la base de datos MySQL, se implementó el sistema de autenticación con JWT y se preparó la pantalla de login.

Iteración 2: Backend de conversaciones y eventos WebSocket. Se implementaron todos los endpoints relacionados con las conversaciones (listado con filtros, asignaciones, estados, mensajes), la integración con el proveedor de mensajería mediante webhooks, y el sistema de eventos en tiempo real con Socket.IO.

Iteración 3: Frontend del listado de conversaciones. Se desarrolló la vista principal con el layout de dos columnas, el listado de conversaciones con filtros, el hilo de mensajes con paginación por cursor, las burbujas estilo WhatsApp, la cabecera con asignación y cambio de estado, y la integración con los eventos WebSocket para la actualización en tiempo real.

Iteración 4: Backend del perfil de usuario. Se implementaron los endpoints para obtener el perfil del usuario autenticado y para la subida de avatar con validación de formato y tamaño.

Iteración 5: Frontend del perfil de usuario. Se desarrolló la vista de perfil con la información del usuario, la subida de avatar con drag-and-drop, y el selector de tema claro/oscuro.

Iteración 6: Backend del CRUD de usuarios. Se implementaron los endpoints de listado, creación, edición y desactivación de usuarios con control de acceso exclusivo para administradores.

Iteración 7: Frontend del CRUD de usuarios. Se desarrolló la tabla de usuarios con modales de creación y edición, validación de formularios y feedback visual.

Iteración 8: Backend del dashboard. Se implementaron los endpoints de estadísticas con filtros de rango de fechas, KPIs, métricas por agente y drill-down a conversaciones.

Iteración 9: Frontend del dashboard. Se desarrolló el panel de estadísticas con tarjetas KPI, gráficos de dona y líneas (Chart.js), tabla de rendimiento por agente con drill-down, y adaptación responsiva.

Iteración 10: Backend del sistema de etiquetas. Se implementó el modelo Label y la tabla junction ConversationLabel en Prisma, los endpoints CRUD en /labels con control de acceso ADMIN, y los endpoints POST/DELETE /conversations/:id/labels con validación del límite de 5 etiquetas por conversación. Se añadió el endpoint GET /stats/labels para las métricas de etiquetas en el dashboard. Borrado lógico con CASCADE sobre ConversationLabel.

Iteración 11: Frontend del sistema de etiquetas. Se desarrolló la vista LabelsView.vue con la tabla de gestión de etiquetas y el modal LabelFormModal.vue (nombre + paleta de 12 colores HEX con preview). Se integró el multiselector de etiquetas en ConversationHeader.vue y la visualización de puntos de color en ConversationItem.vue. Se creó el store Pinia labels.js para la caché de etiquetas disponibles.

Iteración 12: Análisis de sentimientos. Se implementó sentiment.service.js con un diccionario propio en español (~60 frases multipalabra y ~100 palabras individuales) y escala de puntuación de -5 a +5 con cuatro categorías (angry, negative, neutral, positive). Se añadió el campo sentiment al modelo Message mediante migración de Prisma. El análisis se aplica automáticamente a todos los mensajes entrantes (direction=IN) al procesarlos en el webhook. Se añadió el badge de sentimiento en MessageBubble.vue. El análisis primero limpia el texto (minúsculas, sin tildes), después busca expresiones de varias palabras como “llevo días esperando” o “muchas gracias”, y por último compara las palabras sueltas con el diccionario, sumando la puntuación final que determina la etiqueta.

Iteración 13: Asistente de datos con IA. Se implementó assistant.service.js con integración a RunPod para Text-to-SQL: el servicio construye un contexto dinámico con las etiquetas y usuarios activos, envía la pregunta al LLM para obtener un SELECT SQL, valida y sanitiza la consulta (bloqueando INSERT/UPDATE/DELETE/DROP), la ejecuta en MySQL vía

Prisma y envía los resultados al LLM para obtener una respuesta en lenguaje natural. Se desarrolló AssistantPanel.vue en el dashboard con historial de conversación, chips de sugerencias e indicador de estado cold-start.

Planificación y presupuesto de horas

El proyecto se ha desarrollado con una dedicación total aproximada de 126 horas, distribuidas de la siguiente manera y en el mismo orden en el que se fueron implementando las funcionalidades:

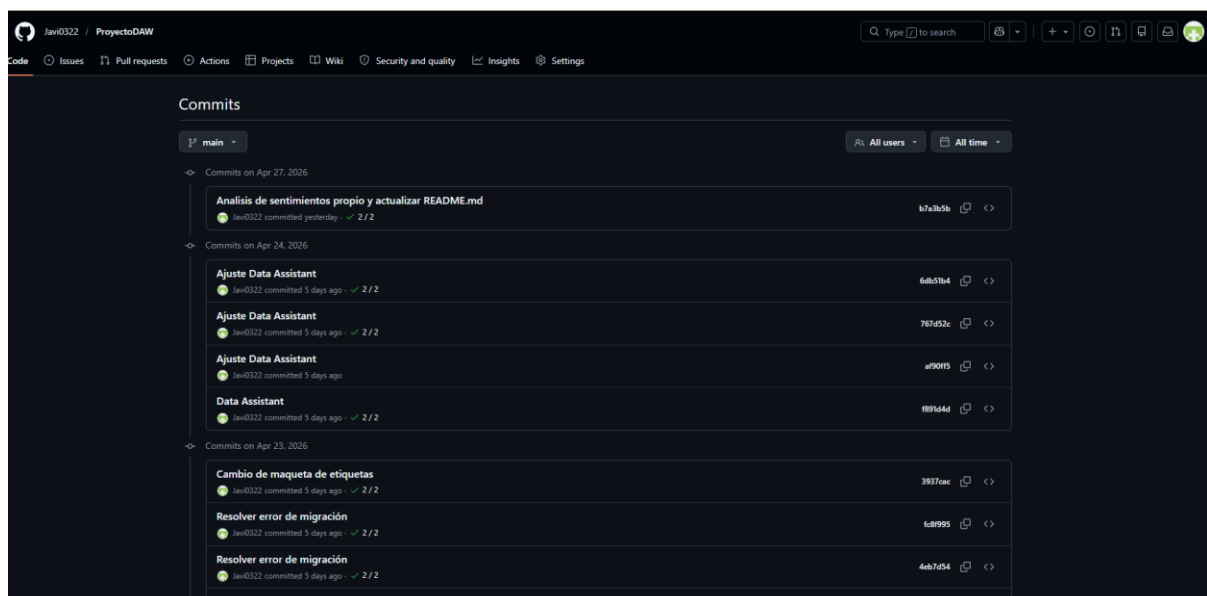
Fase / Tarea	Horas estimadas	Horas reales	Desviación
Análisis y diseño	8	8	0h
Configuración del entorno y autenticación	3	3	0h
Backend: Conversaciones, mensajes, webhooks y WebSocket	20	22	+2h
Frontend: Bandeja de conversaciones y tiempo real	20	22	+2h
Backend: Perfil de usuario	4	4	0h
Frontend: Perfil de usuario	4	4	0h
Backend: CRUD de usuarios	6	6	0h
Frontend: CRUD de usuarios	6	6	0h
Backend: Dashboard y estadísticas	6	6	0h
Frontend: Dashboard y estadísticas	8	8	0h
Responsividad y adaptación móvil	5	4	-1h
Despliegue (Railway, Vercel, Cloudflare)	4	3	-1h
Pruebas y corrección de errores	4	4	0h
Documentación	2	2	0h
Backend: Sistema de etiquetas	6	6	0h
Frontend: Sistema de etiquetas	6	6	0h
Análisis de sentimientos	4	4	0h
Asistente IA (backend + frontend)	10	10	0h
TOTAL	126	128	+2h

Tabla 4: Presupuesto de horas del proyecto

Las principales desviaciones se han producido en la parte de conversaciones, tanto en el backend como en el frontend, que resultó más compleja de lo previsto debido a la lógica de filtrado por roles, la paginación por cursor y la integración completa de los eventos WebSocket. Estas desviaciones se compensaron parcialmente con un menor tiempo del esperado en la adaptación responsive (Tailwind CSS facilitó notablemente esta tarea) y en el despliegue (ya tenía experiencia previa con Railway y Vercel).

README y GIT

El código fuente del proyecto se gestiona con Git, con un repositorio público (García Manotas, 2026) que incluye el código del backend y del frontend en la misma estructura. El fichero README.md contiene instrucciones de instalación, configuración de variables de entorno y comandos para ejecutar el proyecto en desarrollo y producción. El repositorio también incluye, en una carpeta postman/, la colección de peticiones de la API que he usado durante el desarrollo, lista para importar; y un script de seed que crea automáticamente un usuario administrador inicial para poder acceder al sistema desde el primer momento.



TRABAJOS FUTUROS

A continuación se describen las líneas de ampliación y mejora proyectadas para futuras versiones de PowerChat:

Soporte multimedia: añadir la capacidad de enviar y recibir imágenes, vídeos, documentos y notas de voz, además de mensajes de texto. Esto requiere ampliar el modelo de datos de mensajes y la integración con el proveedor de mensajería para soportar diferentes tipos de contenido.

Respuestas rápidas y plantillas: implementar un sistema de respuestas predefinidas que los agentes puedan utilizar para agilizar la atención al cliente. Incluiría un editor de plantillas con variables dinámicas (nombre del cliente, número de referencia, etc.).

Notas internas: permitir que los agentes añadan notas privadas a las conversaciones, visibles solo para el equipo interno, para facilitar la coordinación y el traspaso de información entre agentes.

Automatizaciones y chatbots: integrar un sistema de respuestas automáticas para gestionar las primeras interacciones con los clientes fuera del horario de atención o mientras no haya agentes disponibles.

Integración con múltiples canales: ampliar el sistema para soportar otros canales de comunicación además de WhatsApp, como Telegram, Facebook Messenger o Instagram Direct, convirtiendo PowerChat en una plataforma omnicanal.

Mejoras en el dashboard: añadir métricas de tiempo de respuesta, satisfacción del cliente, y exportación de informes en formato PDF o Excel.

Tests automatizados: implementar una suite completa de tests unitarios y de integración tanto en el backend (con Jest o Vitest) como en el frontend (con Vue Test Utils y Cypress para tests end-to-end).

CONCLUSIONES

El desarrollo de PowerChat ha permitido aplicar de forma integral las competencias adquiridas durante el Grado Superior en Desarrollo de Aplicaciones Web, abordando un proyecto completo que cubre todas las fases del desarrollo de software, desde el análisis de requisitos hasta el despliegue de una aplicación web funcional.

Desde el punto de vista técnico, el proyecto demuestra que es posible construir una plataforma de gestión multiagente de conversaciones de WhatsApp utilizando tecnologías modernas del ecosistema JavaScript. La combinación de Vue 3 en el frontend y Node.js con Express en el backend ha permitido desarrollar una solución coherente y mantenible, mientras que la comunicación en tiempo real aporta una experiencia cercana a la de herramientas profesionales utilizadas en entornos empresariales.

Además de las funcionalidades principales de gestión de conversaciones, el proyecto incorpora elementos que aportan valor añadido al sistema. El uso de etiquetas permite organizar la información de forma flexible, mientras que el análisis de sentimiento introduce una primera capa de interpretación sobre los mensajes de los clientes.

Como elemento diferencial, se ha integrado un asistente basado en inteligencia artificial que permite realizar consultas en lenguaje natural sobre los datos del sistema. Esta funcionalidad no solo facilita el acceso a la información, sino que introduce una forma más intuitiva de analizar la actividad del negocio. De este modo, PowerChat no se limita a gestionar conversaciones, sino que también permite obtener una visión más clara de lo que está ocurriendo en el canal de WhatsApp, sin necesidad de recurrir a informes tradicionales.

Uno de los aspectos más relevantes del proyecto ha sido la capacidad de integrar múltiples tecnologías en un único sistema funcional, incluyendo frontend, backend, base de datos, comunicación en tiempo real y servicios externos. Asimismo, la experiencia profesional

previa en el sector ha permitido orientar el desarrollo hacia necesidades reales, aportando un enfoque práctico al diseño de la aplicación.

En comparación con soluciones comerciales existentes, PowerChat reproduce los elementos fundamentales de este tipo de plataformas, demostrando cómo pueden implementarse desde cero. Aunque estas herramientas suelen incluir funcionalidades adicionales y mayor grado de optimización, el proyecto desarrollado cumple con los objetivos planteados y ofrece una base sólida sobre la que seguir evolucionando.

En conjunto, el resultado es una aplicación funcional, segura y responsiva que refleja las competencias necesarias para el desarrollo de aplicaciones web completas, incorporando además elementos diferenciales como el análisis de datos mediante inteligencia artificial. El proyecto demuestra no solo la capacidad técnica para construir el sistema, sino también la comprensión del problema que resuelve y su aplicación en un contexto real.

El sistema se ha desarrollado con un alcance controlado, priorizando la funcionalidad y la claridad del diseño frente a la complejidad técnica avanzada.

REFERENCIAS

- Auth0. (s. f.). *JSON Web Tokens introduction*. <https://jwt.io/introduction>
- Chart.js. (s. f.). *Chart.js documentation*. <https://www.chartjs.org/docs/latest/>
- García Manotas, J. (2026). *PowerChat* [Repositorio de software]. GitHub.
<https://github.com/Javi0322/ProyectoDAW>
- OpenJS Foundation. (s. f.). *Node.js documentation*. <https://nodejs.org/docs/latest/api/>
- Oracle Corporation. (s. f.). *MySQL 8.0 reference manual*.
<https://dev.mysql.com/doc/refman/8.0/en/>
- Pinia. (s. f.). *Pinia documentation*. <https://pinia.vuejs.org/>
- Prisma. (s. f.). *Prisma documentation*. <https://www.prisma.io/docs>
- RunPod. (s. f.). *RunPod documentation*. <https://docs.runpod.io/>
- Socket.IO. (s. f.). *Socket.IO documentation*. <https://socket.io/docs/v4/>
- StrongLoop/Express. (s. f.). *Express.js documentation*. <https://expressjs.com/>
- Tailwind Labs. (s. f.). *Tailwind CSS documentation*. <https://tailwindcss.com/docs>
- Vite. (s. f.). *Vite documentation*. <https://vitejs.dev/guide/>
- Vue.js. (s. f.). *Vue.js 3 documentation*. <https://vuejs.org/guide/introduction.html>
- Whanmo. (s. f.). *Whanmo – WhatsApp multiagente para empresas*. <https://whanmo.com/>



PowerChat

Javier García Manotas